

학습 기법

5장에서 다루는 내용

- 데이터로부터 알고리즘이 학습하는 방법
- 미분과 경사 하강을 사용한 파라미터 추정이라는 관점으로 학습에 대한 재구성
- 간단한 학습 알고리즘 훑어보기
- 학습을 돕는 파이토치의 자동미분

지난 십여 년간 일어난 머신러닝 붐으로 경험에서 배우는 기계라는 개념은 기술계와 언론계 모두의 주된 관심사가 되어 왔다. 머신러닝, 즉 기계가 학습한다는 것은 정확히 어떤 것일까? 머신러닝의 구체적인 동작 방식은 무엇일까? 뒤편에 숨어 있는 알고리즘은 어떤 것일까? 관찰자의 입장에서 학습 알고리즘은 입력과 이에 대응하는 출력 쌍으로 표현된다. 한 번 학습이 되면 알고리즘은 학습 때 보았던 입력 데이터와 충분히 유사한 새로운 데이터가 들어올 때 정확한 출력을 만들어낸다. 또 딥러닝에서는 심지어 출력 형태가 입력 형태와 현저하게 달라도 동작한다. 예를 들면, 2장에서 살펴본, 이미지를 입력받고 이를 설명하는 문장을 출력하는 경우처럼 다양한 분야에서도 찾아볼 수 있다.

5.1 시대를 초월하는 모델링 학습

입출력 관계를 설명하는 모델 구축의 기원은 수 세기 전으로 거슬러 올라간다. 독일의 수학 천문학자인 요하네스 케플러(Johannes Kepler, 1571-1630)는 1600년대 초 행성의 운동에 대한 세 가지 법칙을 발견하는데, 이 법칙은 그의 멘토인 튀코 브라헤(Tycho Brahe)가 맨눈으로 밤하늘을 관찰한 결과를 종이에 적은 데이터에 기반했다. 뉴턴의 중력 법칙(케플러의 업적은 이후 뉴턴에 영향을 미쳤다)이 없던 그 시절, 케플러는 데이터에 부합하면서도 최대한 단순한 기하 모델을 추론하려 노력했다. 부연하자면, 케플러는 뭔가 이치에 잘 맞지 않은 데이터를 6년간이나 관찰하며 깨달음을 얻어낸 뒤에 마침내 3가지 법칙을 만들어냈다.¹ 그림 5.1에서 이 과정을 볼 수 있다.

케플러의 제1 법칙은 “모든 행성의 궤도는 타원이며 두 초점 중 하나에 태양이 위치한다.”이다. 케플러는 궤도가 왜 타원인지는 모른 채 행성이나 큰 행성(목성 같은)의 위성을 관찰하여 타원 궤도의 모양(이심률)과 크기(통경)²를 측정했다. 그리고 데이터를 기반으로 이 두 파라미터를 계산해서 행성의 천체 이동 경로를 파악할 수 있었다. 제2 법칙인 “행성과 태양을 이은 선분이 같은 시간 동안 지나면서 만드는 면적은 일정하다.”를 발견한 후에는 관측을 통해 행성이 언제 어떤 지점에 머무를 것인지도 예측할 수 있었다.³

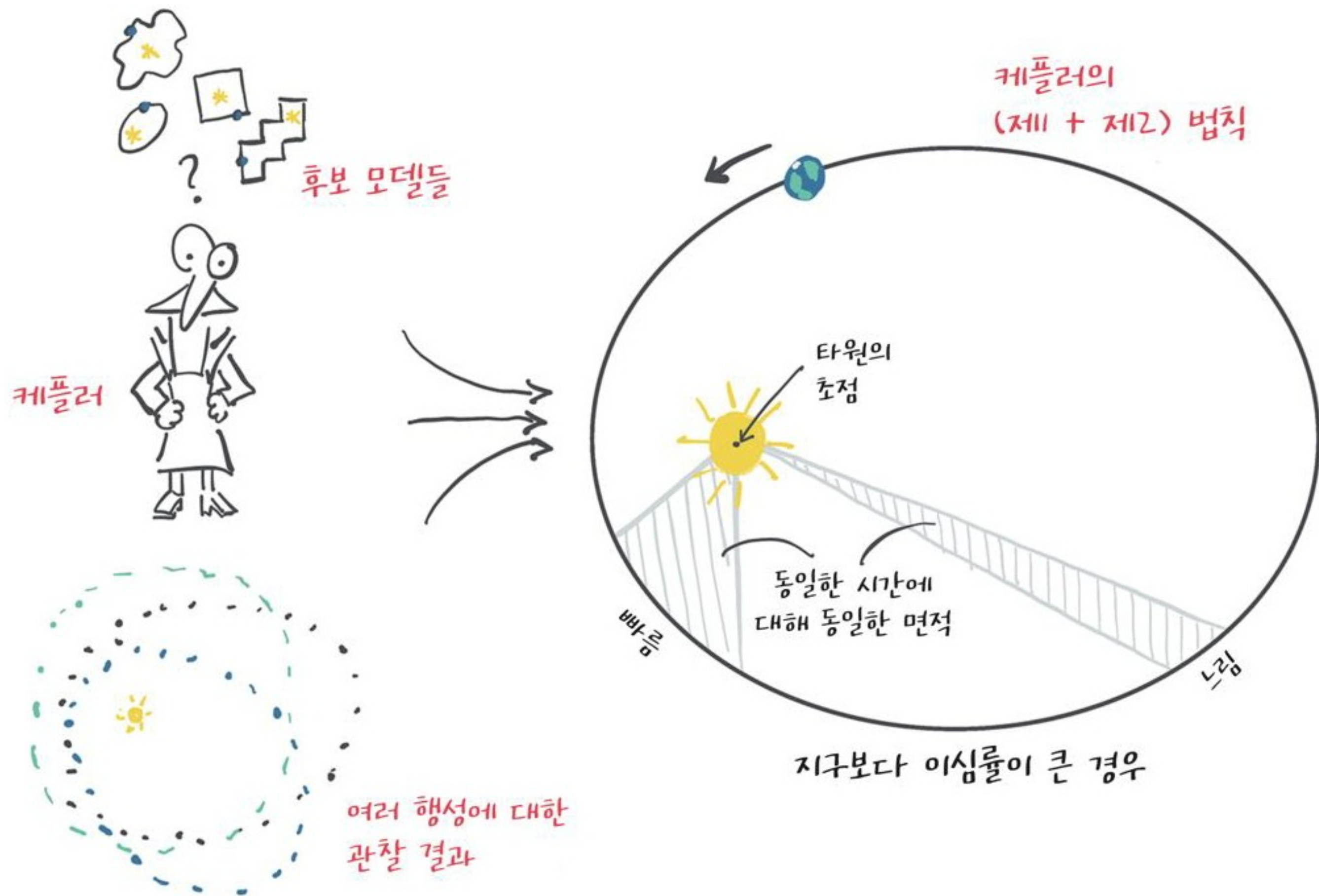


그림 5.1 요하네스 케플러는 여러 후보 모델을 검토하며 데이터에 부합하는 타원을 모델로 정했다.

컴퓨터나 휴대용 계산기 혹은 미적분학도 없던 시절에 케플러는 어떻게 타원의 이심률^{離心率}과 크기를 측정할 수 있었을까? 케플러의 회고를 살펴볼 수 있는 그의 저서 『신천문학』이나 J. V. 필드^{Field}의 글인 “증명의 기원들”(http://mng.bz/9007)에서 이를 찾아볼 수 있다.

케플러는 여러 관찰을 통해 다양한 모양을 시도해 마침내 곡선을 찾아냈고, 이로부터 더 많은 위치를 발견할 수 있게 되었다. 그리고 나서 계산된 위치가 관찰된 위치와 동일한지 점검했다.

– J. V. 필드

6년 동안 케플러가 이뤄낸 업적을 요약하면 다음과 같다.

1. 동료 브라헤로부터(힘들이지 않고) 좋은 데이터를 많이 얻었고,
2. 수상한 점을 느껴서 이를 가시화해보려고 시도했으며,
3. 데이터에 가장 잘 부합할 만한 최대한 단순한 모델(타원)을 골라서,
4. 데이터를 쪼개서 일부만 사용하고 나머지는 검증을 위해 남겨뒀다.
5. 잠정적으로 타원에 대한 이심률과 크기를 정하고 이 모델이 관찰 결과와 맞을 때까지 반복했으며,
6. 별도의 관찰을 통해 자신의 모델을 검증한 후
7. 의심스러운 부분을 되돌아보았다.

이와 같은 과정은 1609년에서 날아온, 일종의 ‘데이터과학 핸드북’이나 마찬가지다. 과학의 역사는 이 일곱 가지 단계를 기반으로 만들어졌으며 수 세기 동안 이 단계를 벗어나는 것은 재앙에 이르는 지름길임을 우리는 배워왔다.⁴

이 과정은 우리가 데이터로부터 무언가를 배울 때 정의하는 순서와 정확하게 일치한다. 실제로, 이 책에서 데이터에 **맞춰**fitting 가는 과정과 데이터로부터 **학습**learning하는 알고리즘을 만드는 과정도 이와 같다. 그리고 여기에는 항상 데이터로부터 평가되는 알 수 없는 파라미터를 가진 함수가 있음을 전제한다. 이와 같은 함수를 ‘모델’이라 일컫는다.

데이터로부터 배운다는 것은 기반 모델이 (케플러의 타원처럼) 특정한 문제를 풀기 위해 만들어진 것이 아님을 전제하며, 더 다양한 종류의 함수 중 하나로 근사할 수 있는 것 아닌가라고 생각할 수도 있을 것이다. 케플러가 직관을 발휘해 타원을 상정하고 데이터를 이에 맞춰가는 것과 달리 신경망은 이런 통찰력 없이도 튀코 브라헤의 궤도를 잘 예측할 수 있다. 하지만 이랬다면 아마 아이작 뉴턴은 일반화된 모델로부터 중력 법칙을 발견해내느라 매우 고생했을 것이다.

이 책에서 우리가 관심 있어 하는 모델은 특정한 범위에 한정된 작업을 수행하기 위해 만들어진 모델이 아니라 입출력 쌍을 활용한 다양한 유사 작업에 대해 스스로를 최적화하기 위해 자동으로 적응하는 모델이다. 바꿔 말하면 특정 작업에 대한 데이터로 학습한 일반화된 모델이다. 파이토치는 파라미터 관점에서 잔차^{fitting error}(또는 residual)의 미분을 분석적으로 표현하기 위한 모델을 쉽게 생성하도록 설계되어 있다. 지금은 이 문장이 잘 이해되지 않더라도 걱정하지 말자. 다음 절 전반에 걸쳐 이 부분을 확실하게 이해할 수 있을 것이다.

이 장에서는 일반 함수의 적합^{fitting}을 자동화하는 방법에 대해 다룬다. 사실 이것이 바로 딥러닝의 핵심이다. 심층 신경망이 우리가 말하는 일반 함수며, 파이토치는 이 과정을 최대한 단순하고 투명하게 만들어준다. 핵심 개념을 제대로 이해하기 위해 먼저 심층 신경망보다 훨씬 단순한 모델로 시작해보자. 이를 통해 이 장에서 설명하는 첫 번째 원리로부터 학습 알고리즘의 동작 방식을 이해하고, 다음 6장에서 더 복잡한 모델로 넘어가자.

5.2 학습은 파라미터 추정에 불과하다

이 절에서는 새로운 데이터에 대해 잘 예측할 수 있도록 어떻게 데이터를 얻고, 모델을 고르고, 모델의 파라미터를 추정할지를 배운다. 복잡한 행성의 움직임은 잠시 접어두고 물리학의 두 번째 어려운 문제인 계측 조정으로 넘어가자.

그림 5.2는 이 장의 마지막에서 우리가 구현할 내용을 개략적으로 그려본 것이다. 입력 및 입력에 대응하는 출력인 실측 자료^{ground truth}와 가중치 초기값이 주어졌을 때, 모델에 입력 데이터가 들어가고(순방향 전달) 실측값과 출력 결과값을 비교해서 오차를 계산한다. 그리고 모델의 파라미터(가중치)를 최적화하기 위해 가중치를 오차값에 따라 일정 단위만큼(파라미터별 오류의 기울기^{gradient}로 보면 된다) 변경한다. 이 변경값은 합성 함수(역방향 전달)의 미분값을 연속으로 계산하는 규칙^{chain rule}을 통해 정해진다. 이렇게 하면 오차가 줄어드는 방향으로 가중치값이 조정된다. 이 과정은 학습 때 사용하지 않았던 데이터에 대한 출력값과 실측값과의 오류가 일정 수준 이하로 떨어질 때까지 반복된다. 지금은 이 내용이 이해가 잘 안 되더라도 이 장 전체에서 상세히 설명하니 걱정하지 말자. 이 장의 마지막에서는 모든 부분이 연결되면서 여기서 설명한 내용이 완벽하게 이해될 것이다.

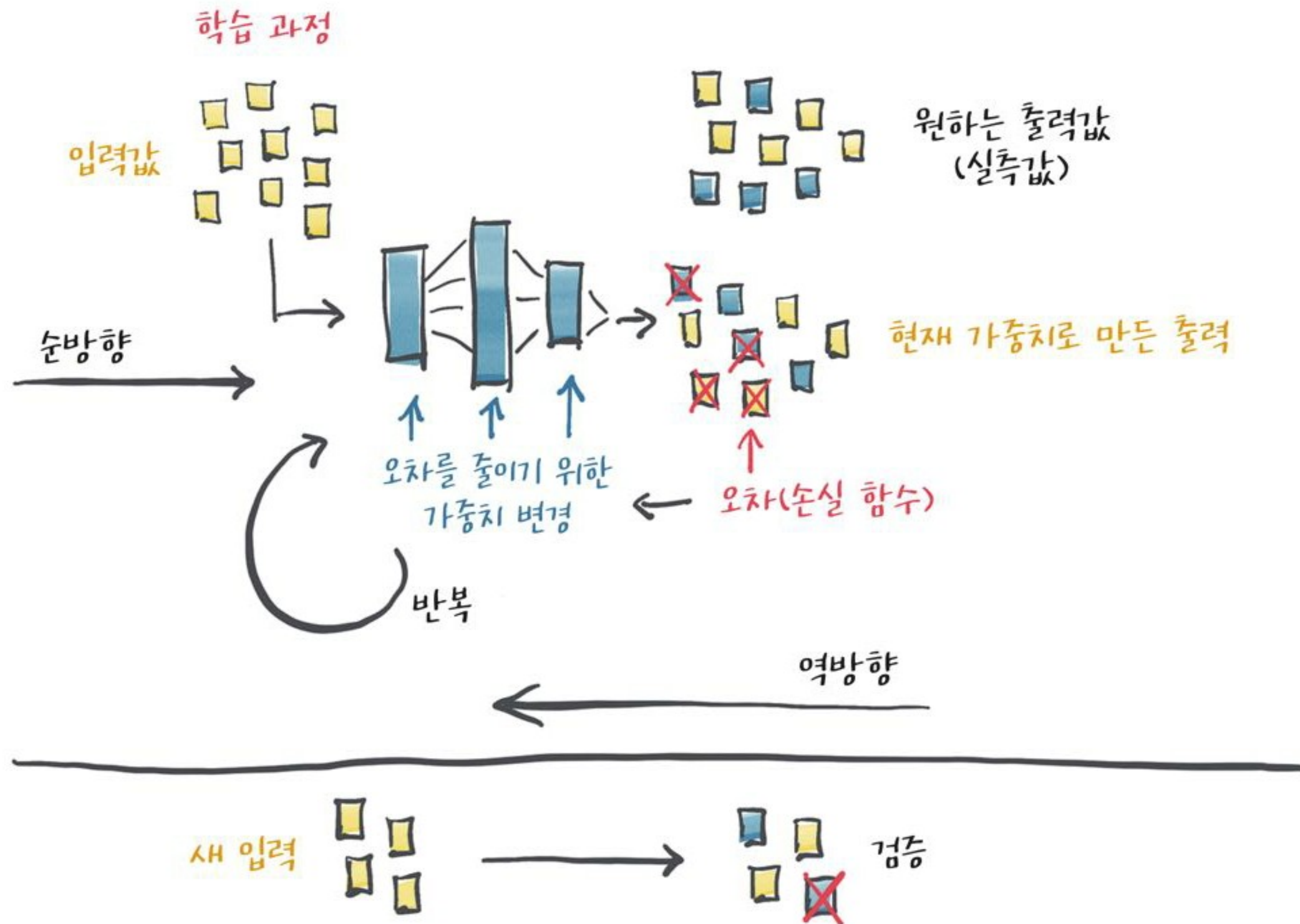


그림 5.2 학습 과정의 멘탈 모델

이제 약간의 노이즈가 섞인 데이터셋이 있는 문제에 대해 모델을 만들고 해결을 위한 학습 알고리즘을 만들어보자. 시작하는 입장이니 모든 부분을 직접 만들게 되지만, 마지막에 가서는 파이토치가 대신하게 만들 것이다. 예제가 매우 단순하고 (아직은!) 모델도 신경망이라 부르기에는 다소 모호하더라도 이 장을 끝낼 즈음에는 심층 신경망 훈련의 필수 개념을 이해하게 될 것이다.

5.2.1 온도 문제

자, 어디론가 여행을 다녀오면서 멋스러운 벽걸이 아날로그 온도계를 하나 구입했다. 정말 훌륭해 보이고 거실에 환상적으로 잘 어울린다. 한 가지 문제는 온도계에 단위가 없다는 점이다. 하지만 우리에게겐 계획이 있으니 걱정하지 말자. 우리가 선호하는 단위를 사용하여 온도계를 읽고 일치하는 온도를 기록한 데이터셋을 만든 후, 학습을 위한 모델을 하나 골라 오차가 충분히 낮아질 때까지 반복적으로 가중치를 조절해서 마지막에는 우리가 이해하는 온도 단위로 새로운 눈금을 해석할 수 있도록 만들 것이다.⁵

케플러가 사용했던 과정을 똑같이 따라해보자. 대신 그 시절에는 사용할 수 없었던 도구인 파이토치를 사용해 볼 것이다!

5.2.2 데이터 수집

친숙하고 오래된 섭씨⁶ 단위의 온도 데이터 노트를 만들고 새 온도계로 측정하면서 데이터를 수집하자. 몇 주 뒤, 다음과 같은 데이터를 만들었다(`code/p1ch5/1_parameter_estimation.ipynb`).

```
# In[2]:  
t_c = [0.5, 14.0, 15.0, 28.0, 11.0, 8.0, 3.0, -4.0, 6.0, 13.0, 21.0]  
t_u = [35.7, 55.9, 58.2, 81.9, 56.3, 48.9, 33.9, 21.8, 48.4, 60.4, 68.4]  
t_c = torch.tensor(t_c)  
t_u = torch.tensor(t_u)
```

`t_c` 값은 섭씨 온도 값이고 `t_u` 값은 우리가 모르는 단위의 값이다. 이 두 값은 온도계에서 얻었고 우리의 대략적인 판단으로 읽어낸 어느 정도 노이즈가 있는 측정값이다. 편의를 위해 데이터를 미리 텐서에 넣어 뒀다.

5.2.3 데이터 시각화

간략하게 만든, 그림 5.3의 차트를 보면 노이즈는 조금 있지만 일정한 패턴을 확인할 수 있다.

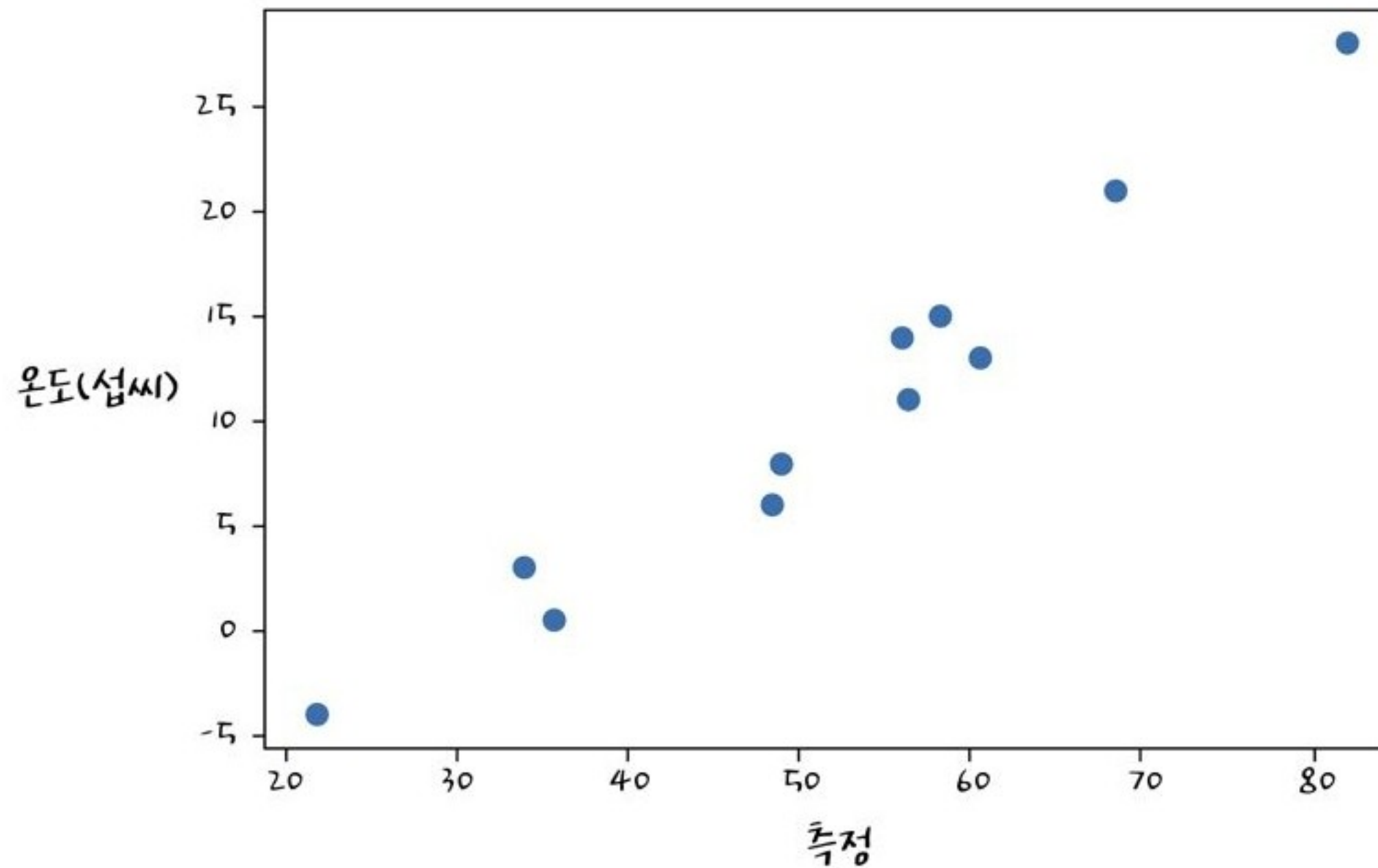


그림 5.3 우리가 만든 데이터는 선형 모델을 따르는 것 같다.

노트

스포일러 주의: 문제와 데이터를 조작했으니 우리는 이미 선형 모델임을 알고 있다. 이 점은 조금 뒤에 확인해볼 테다. 하지만 여기서 중요한 점은 이 문제가 파이토치의 동작 방식을 이해하는 데 유용한 예제라는 사실이다.

5.2.4 먼저 선형 모델을 골라 시도해보기

답을 모른다고 가정하고 케플러처럼 두 측정 값 사이에서 변환하는 역할을 해줄 모델 중 가장 단순한 것을 가정하자. 둘 사이에 선형 관계가 존재할지 모른다. 즉 t_u 에 어떤 값을 곱하고 상수를 더하면 섭씨 단위의 온도값을 얻을지 모르겠다(오차를 무시해도 될 수준에서).

$$t_c = w * t_u + b$$

그럴듯한 가정인가? 아마도. 최종 모델이 얼마나 잘 동작할지는 해보면 될 일이다. 가중치^{weight}와 편향값^{bias}을 각각 w 와 b 로 부르자. 이 두 용어는 이미 선형 스케일링(확대/축소)에서 일반적으로 사용하는 표현이며 앞으로도 계속 접할 것이다.⁷

이제 우리 모델의 파라미터인 w 와 b 를 데이터 기반으로 추정해야 한다. 이렇게 함으로써 모델을 통해 t_u 로 표기한 알 수 없는 온도값을 섭씨 단위로 측정한 실측 값에 최대한 가깝도록 만들게 된다. 여러 번의 측정을 통해 선에 제대로 맞춰가는 것 같다면 잘 이해한 것이다. 우리의 의도가 정확히 구현되고 있다! 파이토치를 사용해서 이 단순한 예제를 풀어보면서 신경망 훈련은 결과적으로 이보다 약간 많은 (혹은 훨씬 더 많은) 파라미터를 가진 모델을 조금씩 정교하게 바꿔나가는 과정임을 이해하자.

내용을 다시 정리해보면, 일단 우리에게 알 수 없는 파라미터를 가진 모델이 있고 이 파라미터의 값을 잘 추정해서 측정된 값과 예측값인 출력값 사이의 오차가 최대한 작아지게 만든다. 오차 측정을 어떻게 할지 구체적으로 정의해야 하는데, **손실 함수** loss function라고 불리는 측정 함수를 만들되 오차가 높으면 함수의 출력값도 높아지도록 정의하여 완벽하게 일치할 경우에는 함수의 출력값을 가능한 한 작게 만들면 된다. 이렇게 손실 함수의 값이 최소인 지점에서 w 와 b 를 찾는 과정을 최적화 과정이라 한다.

5.3 손실을 줄이기 위한 방안

손실 함수(비용 함수라고도 부른다)는 학습 과정이 최소화하고자 하는 단일 값을 계산하는 함수다. 손실 함수는 일반적으로 훈련(training) 샘플로부터 기대하는 출력값과 모델이 샘플에 대해 실제 출력한 값 사이의 차이를 계산한다. 우리의 경우에서 손실 함수는 모델이 출력한 온도인 t_p 와 예측한 값과의 차이인 $t_p - t_c$ 이다.

여기에서 손실 함수는 t_p 가 t_c 보다 크든 작든 간에 항상 양수의 차이가 나오게 해서 t_p 가 t_c 로 맞춰가는 데 사용할 수 있게 해야 한다. 이를 위해 간단하게 $|t_p - t_c|$ 혹은 $(t_p - t_c)^2$ 를 사용할 수 있다. 우리가 어떤 형태를 사용하는지에 따라 특정 오차를 강조하거나 무시하게 된다. 개념적으로 손실 함수는 훈련 샘플로부터 나오는 오차 중 어떤 경우를 우선해서 보완해 나갈 것인지를 결정하게 된다. 즉 파라미터 조정은 손실값이 적은 샘플의 출력을 변경하기보다 가중치가 큰 샘플을 우선적으로 보정하게 된다.

두 손실 함수 모두 최소 오차는 0이고 어느 방향으로든 예측값이 실젯값보다 커지면 오차도 단조적으로 증가한다. 그래프의 상승 기울기 역시 최솟값으로부터 단조적으로 증가하기 때문에 둘 다 볼록^{convex}하다고 말할 수 있다. 모델이 선형이므로 w 와 b 로 만들어진 손실 함수 역시 볼록하다.⁸ 손실이 모델 파라미터의 볼록 함수인 경우는 특수한 알고리즘을 통해 매우 효과적으로 최솟값을 찾을 수 있기 때문에 다루기가 매우 좋다. 이번 장에서는 이보다는 덜 강력하지만 더 일반적으로 적용 가능한 방식을 사용한다. 왜냐하면 우리가 관심 있어 하는 심층 신경망의 손실 그래프는 입력에 대한 볼록 함수가 아니기 때문이다.

그림 5.4에 보이는 두 손실 함수 $|t_p - t_c|$ 와 $(t_p - t_c)^2$ 에서 차이값에 제곱을 한 형태가 최솟값 근처에서 더 잘 동작할 것 같아 보인다. 예를 들어 t_p 가 t_c 인 경우에 손실값의 제곱에 대한 t_p 로의 미분값은 0이 된다. 반면에 절댓값을 사용한 경우는 우리가 수렴하고자 하는 경우에 미분값을 정의할 수 없다. 실제에서는 생각보다 큰 문제가 되지는 않지만, 일단은 차이를 제곱한 손실 함수를 사용하자.

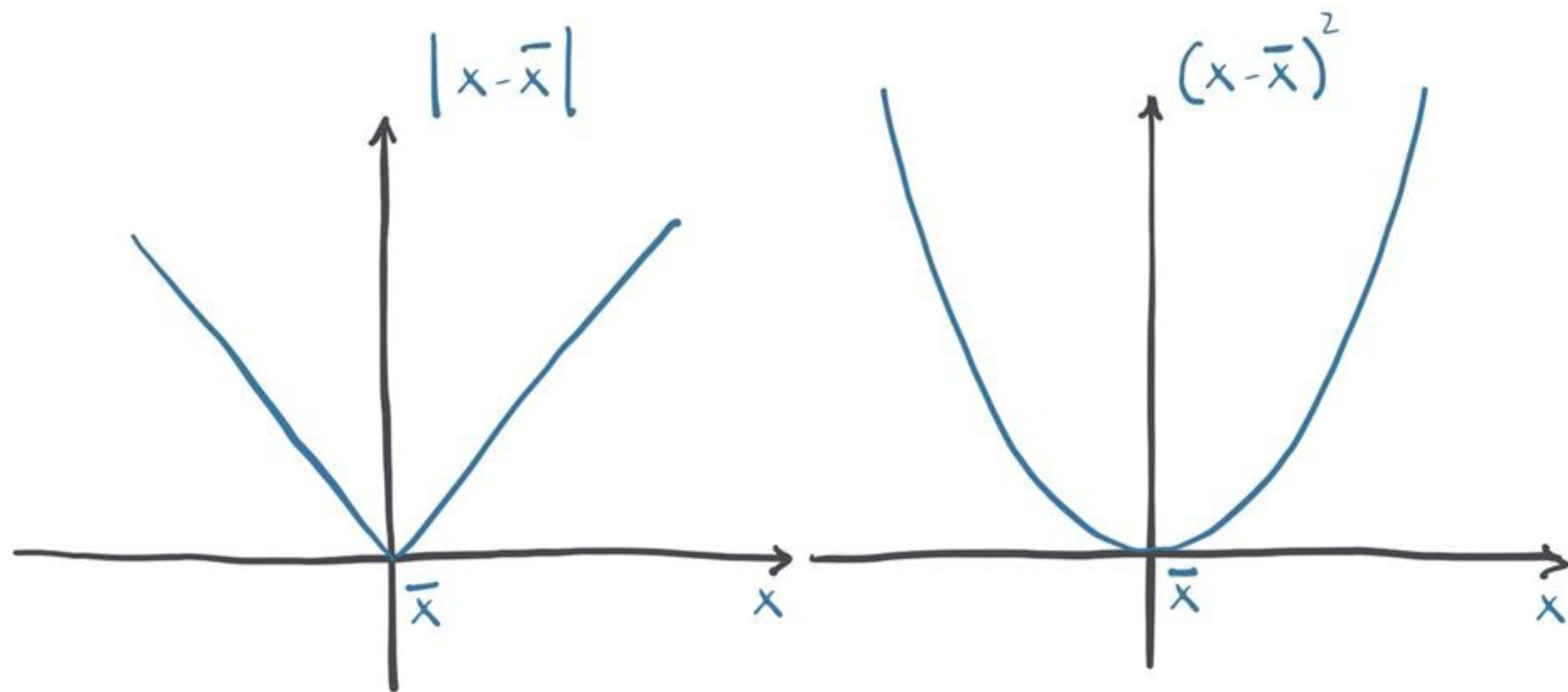


그림 5.4 차이를 절댓값으로 구한 경우와 차이를 제곱한 경우의 비교

제곱을 사용한 차이가 절댓값을 사용한 차이보다 잘못된 결과에 더 많은 불이익을 준다는 사실에 주목할 필요가 있다. 대부분은 빈번하더라도 조금 틀린 값이 많이 나오는 편이 완전히 틀린 값이 어느 정도 나올 때보다 더 바람직한데, 차이를 제곱한 경우는 이런 식으로 오차 보정에 우선순위를 주도록 동작한다.

5.3.1 파이토치로 문제 풀어보기

모델과 손실 함수를 알아봤으니 이미 그림 5.2에서 설명한 고차원의 개념에서 중요한 부분을 이해한 셈이다. 이제 학습 과정을 설정하고 실제 데이터를 넣어보자. 이를 위한 함수 표기도 이제 알고 있으므로 파이토치로 넘어가보자. 결국 재미를 쫓다 보니 여기까지 왔다.

텐서에 이미 데이터를 넣어 놓았으므로 파이썬 함수로 모델을 작성하자.

```
# In[3]:  
def model(t_u, w, b):  
    return w * t_u + b
```

t_u , w , b 는 순서대로 입력 텐서, 가중치 파라미터, 편향값 파라미터다. 우리 모델에서 파라미터는 파이토치 스칼라값(0차원 텐서)이고 행렬곱 연산은 텐서를 반환하기 위해 브로드캐스팅을 사용할 것이다. 손실 함수를 작성하자.

```
# In[4]:  
def loss_fn(t_p, t_c):  
    squared_diffs = (t_p - t_c)**2  
    return squared_diffs.mean()
```

이 코드는 요소 단위로 값의 차이를 제공한 텐서를 만든 후 텐서의 모든 요소에 대한 평균을 구하여 스칼라값을 만들어 내는 손실 함수다. 이를 **평균 제곱 손실** mean square loss이라고 한다.⁹

이제 파라미터를 초기화하고 모델을 호출할 수 있다.

```
# In[5]:  
w = torch.ones(())  
b = torch.zeros(())  
  
t_p = model(t_u, w, b)  
t_p  
  
# Out[5]:  
tensor([35.7000, 55.9000, 58.2000, 81.9000, 56.3000, 48.9000, 33.9000,  
        21.8000, 48.4000, 60.4000, 68.4000])
```

손실값을 체크해보자.

```
# In[6]:  
loss = loss_fn(t_p, t_c)  
loss  
  
# Out[6]:  
tensor(1763.8846)
```

이번 절에서는 모델과 손실을 구현해봤다. 그리고 예제의 핵심인 손실이 최소일 때의 w 와 b 를 어떻게 추정할 것인가에 대한 문제에 도달했다. 이후 과정은 수동으로 먼저 진행한 후, 일반적인 상황에서 바로 사용할 수 있는 파이토치로 해결하는 방법을 알아보자.

3장에서 브로드캐스팅(broadcasting)을 언급하며 필요할 때 더 자세히 살펴보기로 했다. 우리 예제에는 두 개의 스칼라값(0차원 텐서)인 w 와 b 가 있고 이를 길이가 b 인(1차원 텐서) 벡터와 곱셈 및 덧셈을 수행한다.

초기 버전의 파이토치에서처럼 일반적으로 덧셈과 뺄셈, 곱셈, 나눗셈 같은 연산은 동일한 차원 정보를 가진 텐서에 대해 요소 대 요소 단위로 이항 연산만 수행했다. 텐서 내에서 동일한 위치에 있는 요소들끼리의 연산을 수행한 결과값이 출력 텐서의 해당 위치에 들어가는 형태였다.

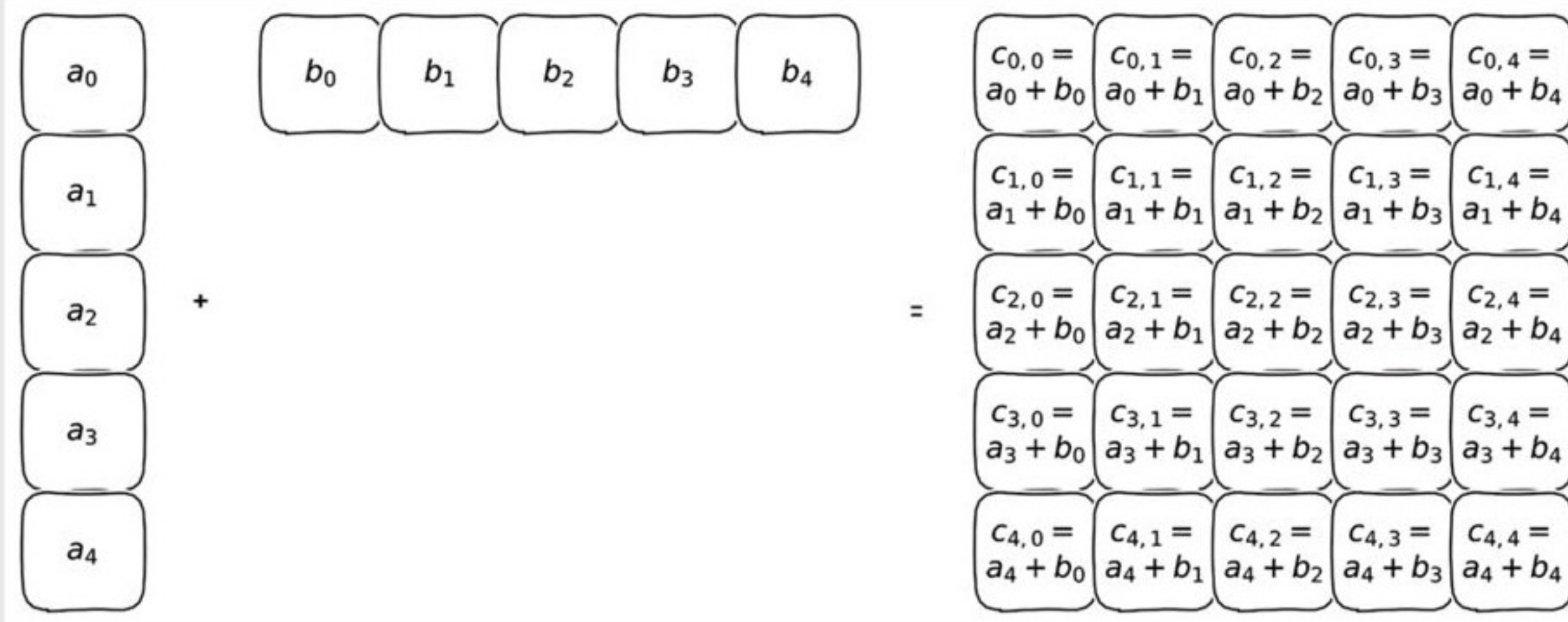
넘파이의 잘 알려진 기능 중 하나인 브로드캐스팅은 파이토치에도 반영됐고 대부분의 이항 연산에 대해 이런 제약을 해소해준다. 브로드캐스팅은 다음과 같은 규칙으로 텐서 요소를 다룬다.

- 차원 인덱스를 뒤에서부터 세면서 만일 해당 차원의 피연산자에 해당하는 차원의 크기가 1이면, 파이토치는 단일 요소를 가진 차원에 대응하는 다른 텐서 차원의 각 요소에 이 차원의 단일 요소를 사용할 것이다.
- 만일 연산 대상인 차원이 둘 다 1보다 크면 둘의 크기는 같아야 하고 일대일로 대응해서 연산한다.
- 만일 한 텐서가 다른 텐서보다 차원이 큰 경우, 차원이 큰 텐서의 각 차원에 대해 차원이 작은 텐서 전체가 대응하여 연산을 수행한다.

꽤 복잡하게 들리지만 (동시에 집중하지 않으면 실수하기도 쉽다. 그래서 3.4절에서 했던 것처럼 차원에 이름을 부여하기도 한다) 실제로는 텐서의 차원을 적어가면서 과정을 살펴보거나 브로드캐스팅을 보기 위해 다음처럼 차원 공간을 그리기도 한다.

몇 가지 코드 예제를 살펴보면 실제로 감이 잡힐 것이다.

```
# In[7]:
x = torch.ones(())
y = torch.ones(3,1)
z = torch.ones(1,3)
a = torch.ones(2, 1, 1)
print(f"shapes: x: {x.shape}, y: {y.shape}")
```



```
print(f"      z: {z.shape}, a: {a.shape}")
print("x * y:", (x * y).shape)
print("y * z:", (y * z).shape)
print("y * z * a:", (y * z * a).shape)
```

```
# Out[7]:
```

```
shapes: x: torch.Size([ ]), y: torch.Size([3, 1])
       z: torch.Size([1, 3]), a: torch.Size([2, 1, 1])
x * y: torch.Size([3, 1])

y * z: torch.Size([3, 3])
y * z * a: torch.Size([2, 3, 3])
```


5.4 경사를 따라 내려가기

이제 경사 하강^{gradient descent} 알고리즘을 사용해 파라미터 관점에서 손실 함수를 최적화한다. 이번 절에서는 첫 번째 규칙인 경사 하강 알고리즘의 동작 방식에 대한 통찰력을 얻어 앞으로 진행 과정에 도움을 받고자 한다. 앞서서도 언급했지만 실상 예제 문제를 더 효과적으로 풀 수 있는 방법들은 많다. 하지만 이 방법들은 대부분의 딥러닝 작업에 적용하기 어렵다. 경사 하강은 매우 단순한 개념이며 수백만 개의 파라미터를 가진 대규모 신경망에도 놀라울 정도로 쉽게 확장 적용할 수 있다.



그림 5.5 사람이 w 손잡이와 b 손잡이를 돌려가며 손실을 줄이는 최적화 작업을 수행하는 장면

그림 5.5를 보고 머릿속에 이미지를 한번 떠올려보자. 두 개의 손잡이가 있는 장치 앞에 서 있고 손잡이에는 각각 w 와 b 가 적혀 있다. 스크린을 통해 손실을 볼 수 있고 그 값을 줄이도록 요청 받았다. 손잡이를 돌리는 행위가 손실 변화에 얼마나 영향을 미칠지는 잘 모르지만, 어떤 경우에 줄어드는지를 확인하며 조작해본다. 그러고 나서 손실이 줄어드는 방향으로 두 손잡이를 돌리기 시작한다. 만일 우리가 최적에서 멀리 떨어져 있다고 가정하면 일단 빨리 값이 줄어들기를 바랄 것이고, 최솟값에 가까워지면 천천히 줄어들면서 값에 가깝게 다가갈 것이다. 그리고 어떤 경우에는 손실이 다시 늘어나는 것을 보고 손잡이를 되돌려서 값이 다시 줄어드는 것을 확인할 수 있다. 손실값이 서서히 변할 때, 다시 올라가지 않도록 손잡이를 미세하게 조정하다가 시간이 지나면 결국 최솟값으로 수렴하게 된다.

5.4.1 손실 줄이기

경사 하강은 방금 설명한 시나리오와 크게 다르지 않다. 각 파라미터와 관련해 손실의 변화율을 계산해 손실이 줄어드는 방향으로 파라미터 값을 바꿔나간다. 손잡이를 만지작거리는 것처럼 w 와 b 값을 아주 조금 늘리거나 줄여서 그 사이에 손실값이 얼마나 변하는지 확인하는 것이다.

```
# In[8]:
delta = 0.1

loss_rate_of_change_w = \
    (loss_fn(model(t_u, w + delta, b), t_c) -
     loss_fn(model(t_u, w - delta, b), t_c)) / (2.0 * delta))
```

이 코드는 현재의 w 와 b 값에서 특정 단위만큼 w 가 증가했을 때의 손실이 변하게 만드는데, 값이 줄어들면 w 를 더 늘려서 손실을 최소화하고, 값이 늘어나면 반대로 w 를 줄여서 손실을 최소화하는 식이다. 그렇다면 w 를 어느 정도로 늘리거나 줄일 것인가? 손실의 변화 비율에 비례해서 w 를 바꾸는 편이 좋다. 이는 손실 함수가 여러 파라미터를 가질수록 더 그렇다. 손실값의 변화에 영향을 많이 미치는 파라미터를 변경한다. 일반적으로 파라미터 값도 서서히 변경하는 것이 좋다. 현재 w 값 근방의 어느 정도 떨어진 곳에서는 변화율이 급격하게 달라질 수 있기 때문이다. 얼마만큼 바뀌갈 것인지에 대한 스케일링 비율을 나타내는 이름은 많은데 머신러닝에서는 주로 `learning_rate`라는 변수명을 사용한다.

```
# In[9]:  
learning_rate = 1e-2  
  
w = w - learning_rate * loss_rate_of_change_w
```

b에 대해서도 동일하게 할 수 있다.

```
# In[10]:  
loss_rate_of_change_b = \  
    (loss_fn(model(t_u, w, b + delta), t_c) -  
     loss_fn(model(t_u, w, b - delta), t_c)) / (2.0 * delta)  
  
b = b - learning_rate * loss_rate_of_change_b
```

이것은 경사 하강에서 기본적인 파라미터 조정을 표현한다. 이런 식의 평가(그리고 충분히 작은 학습률을 고른 상태에서)를 반복함으로써 주어진 데이터에 대해 손실 계산값이 최소로 떨어지는 최적의 파라미터 값으로 수렴하게 되는 것이다. 반복적인 과정을 완료하는 모습도 곧 살펴볼 것이지만, 지금은 변화율이 너무 심해서 조정할 필요가 있으니 더 진행하기 전에 왜 그런지, 또 어떻게 조정하는지를 먼저 알아보자.

5.4.2 분석

w 와 b 인접 영역에서 손실 함수의 동작 방식을 조사하기 위해서 모델과 손실값을 반복적으로 평가하며 변화율을 계산하는 것을 파라미터가 많은 모델로 확장하기란 쉽지 않다. 또 인접 영역을 얼마의 거리까지로 볼 것인지 규정하기도 어렵다. 방금 5.4.1절에서 δ 를 0.1로 뒀다. 하지만 이 값은 w 와 b 의 함수인 손실 함수의 모양에 따라 달라진다. 지정한 δ 값에 비해 손실값이 너무 빠르게 변한다면, 손실값을 최소화하기 위해 어떤 방향으로 파라미터를 조정할지에 대한 판단을 내리기가 쉽지 않을 것이다.

너무 빠르게 변한다면, 손실값을 최소화하기 위해 어떤 방향으로 파라미터를 조정할지에 대한 판단을 내리기가 쉽지는 않을 것이다.

그림 5.6처럼 인접한 거리를 극단적으로 줄이면 어떻게 될까? 이렇게 하는 것은 정확하게 파라미터에 대해 손실 함수를 미분하는 것과 일치한다. 우리가 다루고 있는 것처럼 두 개 이상의 파라미터를 가진 모델에서는 각 파라미터에 대한 손실 함수의 편미분을 구하고 이 편미분 값들을 미분 벡터에 넣자. 이것이 바로 기울기 **gradient**다.

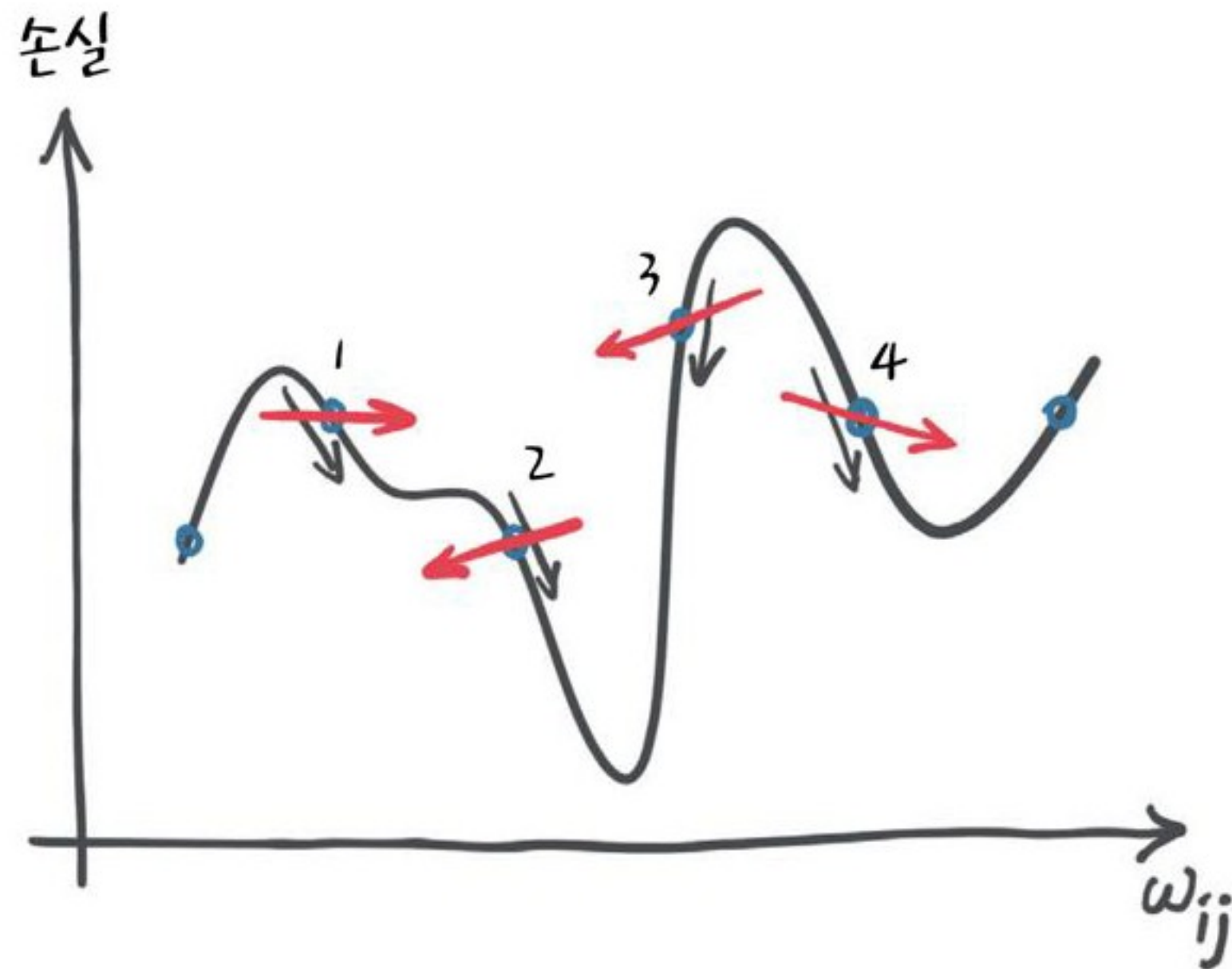


그림 5.6 특정 구간을 기준으로 계산한 기울기와 미분을 통해 계산한 기울기의 차이

■ 미분 계산하기

파라미터에 대한 손실 함수의 미분을 계산하려면 연쇄 규칙을 적용하여 입력(여기에서는 모델의 출력값이다)에 대한 손실 함수의 미분을 계산해서 파라미터에 대한 모델의 미분을 곱할 수 있다.

$$d \text{ loss_fn} / d w = (d \text{ loss_fn} / d t_p) * (d t_p / d w)$$

우리 모델은 선형 함수고 손실 함수는 제곱의 합이다. 미분식을 만들어보자. 손실 함수 식을 기억하면서 다음 코드를 작성하자.

```
# In[4]:
def loss_fn(t_p, t_c):
    squared_diffs = (t_p - t_c)**2
    return squared_diffs.mean()
```

$d x^2 / d x = 2 x$ 를 이용해 다음과 같은 코드를 만들 수 있다.

```
# In[11]:
def dloss_fn(t_p, t_c):
    dsq_diffs = 2 * (t_p - t_c) / t_p.size(0)  ← 평균의 도함수로 나눔
    return dsq_diffs
```

■ 모델에 미분 적용하기

우리 모델은 다음과 같은 코드로 표현된다.

```
# In[3]:  
def model(t_u, w, b):  
    return w * t_u + b
```

그리고 다음과 같이 함수를 정의해서 미분을 구했다.

```
# In[12]:  
def dmodel_dw(t_u, w, b):  
    return t_u
```

```
# In[13]:  
def dmodel_db(t_u, w, b):  
    return 1.0
```


경사 함수 정의하기

모두를 합쳐서 w 와 b 에 대한 손실값의 미분을 반환하는 함수를 만들면 다음과 같다.

```
# In[14]:
```

```
def grad_fn(t_u, t_c, t_p, w, b):  
    dloss_dtp = dloss_fn(t_p, t_c)  
    dloss_dw = dloss_dtp * dmodel_dw(t_u, w, b)  
    dloss_db = dloss_dtp * dmodel_db(t_u, w, b)  
    return torch.stack([dloss_dw.sum(), dloss_db.sum()])
```

합 계산은 파라미터를 모델의
전체 입력 벡터에 대해 적용하던
브로드캐스팅의 역순이다.

그림 5.7에 이 아이디어를 수식으로 나타냈다. 다시 언급하자면, 우리는 손실에 대한 각 편미분을 위한 단일 스칼라 값을 얻기 위해 모든 데이터 포인트에 대해 평균을 구하고 있다.

$$\nabla_{w,b} L = \left(\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b} \right) = \left(\frac{\partial L}{\partial m} \cdot \frac{\partial m}{\partial w}, \frac{\partial L}{\partial m} \cdot \frac{\partial m}{\partial b} \right)$$

손실 $L(m_{w,b}(x))$

기울기

편미분

모델 $m_{w,b}(x)$

파라미터

그림 5.7 가중치에 대한 손실 함수의 미분

5.4.3 모델 적합을 위한 반복

이제 파라미터를 최적화하기 위한 모든 준비가 완료됐다. 파라미터를 위한 임시 값에서 출발해, 고정된 횟수만큼 반복하며 w 와 b 가 변하지 않을 때까지 조정한다. 반복을 중단하기 위한 여러 조건이 있지만 일단은 정한 횟수만큼만 반복하자.

■ 훈련 루프

훈련 과정 중 필요한 몇가지 용어에 대해 더 알아보자. 먼저 훈련 샘플을 가지고 반복적으로 파라미터를 조정하는 훈련의 한 단위를 **에포크**epoch라 부른다.

전체 훈련 루프는 다음과 같다(code/p1ch5/1_parameter_estimation.ipynb).

```
# In[15]:
def training_loop(n_epochs, learning_rate, params, t_u, t_c):
    for epoch in range(1, n_epochs + 1):
        w, b = params

        t_p = model(t_u, w, b)  ← 순방향 전달
        loss = loss_fn(t_p, t_c)
        grad = grad_fn(t_u, t_c, t_p, w, b)  ← 역방향 전달

        params = params - learning_rate * grad

        print('Epoch %d, Loss %f' % (epoch, float(loss)))  ← 로그가 많이 나올 수 있음

    return params
```

출력을 위해 사용하는 실제 로깅 로직은 이보다 더 복잡하지만(노트북 <http://mng.bz/pBB8>의 15번 셀을 참고하라), 이는 이 장의 핵심 개념을 이해하는 데는 중요하지 않으므로 신경쓰지 않아도 된다.

이제 훈련 루프를 실행하자.

```
# In[17]:
training_loop(
    n_epochs = 100,
    learning_rate = 1e-2,
    params = torch.tensor([1.0, 0.0]),
    t_u = t_u,
    t_c = t_c)

# Out[17]:
Epoch 1, Loss 1763.884644
    Params: tensor([-44.1730, -0.8260])
    Grad:  tensor([4517.2969,  82.6000])
Epoch 2, Loss 5802485.500000
    Params: tensor([2568.4014,  45.1637])
    Grad:  tensor([-261257.4219, -4598.9712])
Epoch 3, Loss 19408035840.000000
    Params: tensor([-148527.7344, -2616.3933])
    Grad:  tensor([15109614.0000,  266155.7188])
...
Epoch 10, Loss 90901154706620645225508955521810432.000000
    Params: tensor([3.2144e+17, 5.6621e+15])
    Grad:  tensor([-3.2700e+19, -5.7600e+17])
Epoch 11, Loss inf
    Params: tensor([-1.8590e+19, -3.2746e+17])
    Grad:  tensor([1.8912e+21,  3.3313e+19])

tensor([-1.8590e+19, -3.2746e+17])
```


과도한 훈련

잠깐, 무슨 일이 일어난 걸까? 훈련 과정이 제멋대로 되면서 손실값이 무한대(∞)가 되어버렸다. params 조정이 너무 크다는 신호며, 이런 경우 값이 앞뒤로 진동하면서 조정 값이 점점 커지고, 다음 차례에는 더 심한 과잉 교정으로 이어진다. 최적화 과정은 불안정해지고, 최적화에 수렴하는 대신 발산해 버린다. 그림 5.8처럼 우리가 원하는 것은 조금씩 점차적으로 params 를 조정하는 것이지 점차 값을 키우는 것이 아니다.

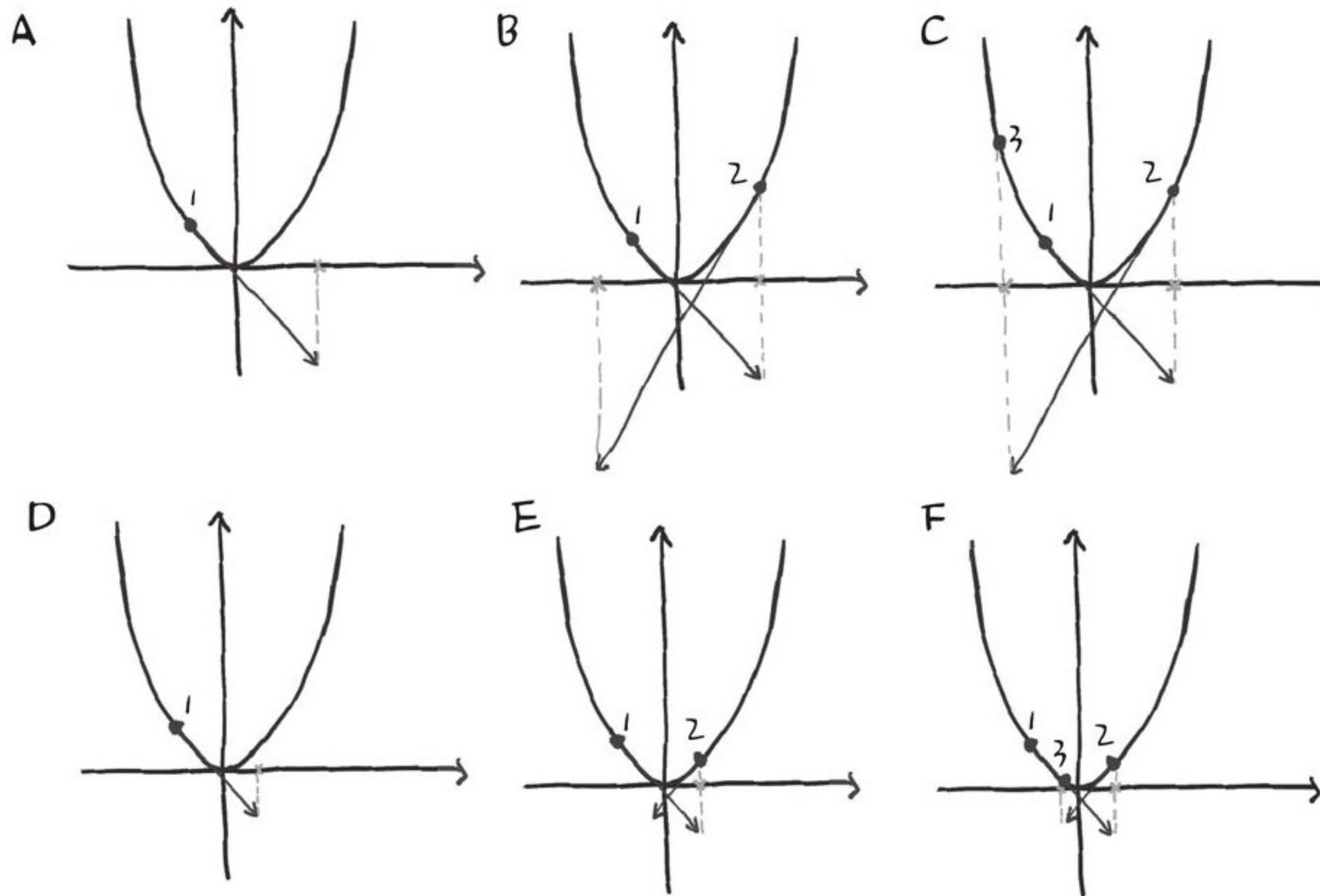


그림 5.8 위: 스텝이 너무 커서 포물선 형의 볼록 함수에서 최적화를 이탈하는 경우. 아래: 작은 스텝으로 최적화에 수렴하는 경우.

어떻게 하면 `learning_rate * grad` 규모를 적절하게 제한할 수 있을까? 사실은 쉽다. `learning_rate` 값이 작으면 된다. 학습률은 우리가 기대한 대로 학습이 되지 않을 때 일반적으로 바꿔볼 수 있는 값 중 하나다.¹⁰ 학습률의 조정은 자릿수 단위로 이뤄지며 `1e-3`이나 `1e-4`로 시작해서 변화량의 자릿수를 줄여나간다. `1e-4`로 놓고 어떻게 되는지 지켜보자.

```
# In[18]:
training_loop(
    n_epochs = 100,
    learning_rate = 1e-4,
    params = torch.tensor([1.0, 0.0]),
    t_u = t_u,
    t_c = t_c)

# Out[18]:
Epoch 1, Loss 1763.884644
    Params: tensor([ 0.5483, -0.0083])
    Grad:  tensor([4517.2969, 82.6000])
```

```
Epoch 2, Loss 323.090546
    Params: tensor([ 0.3623, -0.0118])
    Grad:  tensor([1859.5493, 35.7843])
Epoch 3, Loss 78.929634
    Params: tensor([ 0.2858, -0.0135])
    Grad:  tensor([765.4667, 16.5122])
...
Epoch 10, Loss 29.105242
    Params: tensor([ 0.2324, -0.0166])
    Grad:  tensor([1.4803, 3.0544])
Epoch 11, Loss 29.104168
    Params: tensor([ 0.2323, -0.0169])
    Grad:  tensor([0.5781, 3.0384])
...
Epoch 99, Loss 29.023582
    Params: tensor([ 0.2327, -0.0435])
    Grad:  tensor([-0.0533, 3.0226])
Epoch 100, Loss 29.022669
    Params: tensor([ 0.2327, -0.0438])
    Grad:  tensor([-0.0532, 3.0226])

tensor([ 0.2327, -0.0438])
```


좋다. 안정적으로 동작한다. 하지만 다른 문제가 있다. 파라미터 조정이 너무 조금씩이라 손실이 너무 느리게 줄어들다가 결국 멈춘다. 이 문제는 `learning_rate`를 조정 규모에 따라 변하는 적응형으로 만들어 방지할 수 있다. 이 장의 마지막 부분인 5.5.2절에서 이러한 최적화 전략을 볼 수 있다.

그 외에 업데이트에 대해 잠재적으로 문제가 될 만한 후보가 하나 있다. 그것은 바로 기울기다. 최적화 작업 중에 에포크 1에서 `grad`를 관찰해보자.

5.4.4 입력 정규화

첫 에포크에서 가중치에 대한 기울기는 편향값에 대한 기울기보다 50배나 큰 것을 볼 수 있다. 즉 가중치와 편향값은 범위가 다르다는 이야기다. 이런 경우 하나를 조정하기에 적절한 크기의 학습률은 동시에 다른 값의 업데이트를 불안정하게 만들 수 있다. 반대로 한 파라미터를 업데이트하기에 적합한 학습률은 다른 파라미터 입장에서 충분히 크지 않을 수 있다. 문제의 공식을 바꾸지 않고서는 파라미터를 바꾸기 어려운 상황이다. 파라미터별로 다른 학습률을 줄 수 있겠지만, 파라미터가 많은 모델의 경우 오히려 성가시게 될 수 있고 우리는 이런 식의 세세한 관리를 원하지 않는다.

실은 더 쉽게 제어 가능하다. 입력값을 변경해서 기울기가 서로 큰 차이가 나지 않게 하면 된다. 입력값의 범위가 -1.0에서 1.0 사이를 벗어나지 않도록 바꿔놓으면 가능해진다. 우리 예제에서는 `t_u`에 0.1을 곱하면 이와 유사하게 된다.

```
# In[19]:  
t_un = 0.1 * t_u
```

정규화된 버전의 `t_u`를 의미하는 것으로 변수 이름에 `n`을 붙이자. 이제 정규화된 입력으로 훈련 루프를 돌릴 수 있다.

```
# In[20]:
training_loop(
    n_epochs = 100,
    learning_rate = 1e-2,
    params = torch.tensor([1.0, 0.0]),
    t_u = t_un,  ← t_u 대신 범위를 조정한 t_un을 사용
    t_c = t_c)
```

```
# Out[20]:
Epoch 1, Loss 80.364342
    Params: tensor([1.7761, 0.1064])
    Grad:  tensor([-77.6140, -10.6400])
Epoch 2, Loss 37.574917
    Params: tensor([2.0848, 0.1303])
    Grad:  tensor([-30.8623, -2.3864])
Epoch 3, Loss 30.871077
    Params: tensor([2.2094, 0.1217])
    Grad:  tensor([-12.4631, 0.8587])
...
Epoch 10, Loss 29.030487
    Params: tensor([ 2.3232, -0.0710])
    Grad:  tensor([-0.5355,  2.9295])
Epoch 11, Loss 28.941875
    Params: tensor([ 2.3284, -0.1003])
    Grad:  tensor([-0.5240,  2.9264])
...
Epoch 99, Loss 22.214186
    Params: tensor([ 2.7508, -2.4910])
    Grad:  tensor([-0.4453,  2.5208])
```

```
Epoch 100, Loss 22.148710
    Params: tensor([ 2.7553, -2.5162])
    Grad:  tensor([-0.4446,  2.5165])

tensor([ 2.7553, -2.5162])
```


이제 학습률을 다시 $1e-2$ 로 되돌려 놓아도 반복적인 업데이트 과정에서 파라미터가 발산하지 않는다. 기울기를 살펴보면 둘 다 비슷한 자릿수로 이뤄졌으며, 이 때문에 두 파라미터에 하나의 `learning_rate`를 사용해도 괜찮다. 십 단위로 범위를 단순히 조정하는 대신 제대로 정규화를 진행할 수도 있지만, 지금은 이 정도로도 충분하므로 일단은 이렇게 하자.

노트

정규화는 분명히 신경망 훈련에 도움이 되지만, 이 문제에서 파라미터를 최적화하는 데 반드시 필요한 것은 아니지 않느냐고 반문할 수 있다. 맞는 말이다! 지금 예제는 꽤 작은 문제라서 파라미터를 길들이는 다양한 방법이 존재한다. 하지만 문제가 커지고 복잡해질 경우 정규화는 모델 수렴을 개선하는 데 사용할 수 있는 (결정적인 요소는 아닐지라도) 쉽고 효과적인 방법이다.

params 값의 변화량이 작아질 때까지 루프를 도는 횟수를 충분히 늘려보자. n_epochs에 5,000을 설정한다.

```
# In[21]:
params = training_loop(
    n_epochs = 5000,
    learning_rate = 1e-2,
    params = torch.tensor([1.0, 0.0]),
    t_u = t_un,
    t_c = t_c,
    print_params = False)
```

params

```
# Out[21]:
Epoch 1, Loss 80.364342
Epoch 2, Loss 37.574917
Epoch 3, Loss 30.871077
...
Epoch 10, Loss 29.030487
Epoch 11, Loss 28.941875
...
Epoch 99, Loss 22.214186

Epoch 100, Loss 22.148710
...
Epoch 4000, Loss 2.927680
Epoch 5000, Loss 2.927648

tensor([ 5.3671, -17.3012])
```


좋다. 이제 경사 하강`gradient descent`을 따라 파라미터 값이 바뀌는 과정에서 손실값이 줄어들었다. 정확히 0까지는 줄어들지 않는데 이것은 0에 수렴할 만큼 반복 횟수가 충분하지 않거나 데이터가 선분에 정확하게 걸쳐 있지 않기 때문이다. 예상한 대로 측정이 정확하게 이뤄지지 않은 듯하다. 아니면 값을 부정확하게 읽어서 노이즈가 발생한 것일 수도 있다.

하지만 한번 주목해보자. 찾아낸 w 와 b 값은 섭씨값을 화씨로 변환하는 데 필요한 값과 거의 일치한다(앞서 입력에 0.1을 곱해서 정규화한 부분을 고려해서). 정확한 값은 $w=5.5556$ 과 $b=-17.7778$ 이다. 이제 우리의 멋진 온도계는 항상 화씨로 온도를 보여주게 되었다. 별것 아니지만 이제 경사 하강을 통한 최적화 작업이 실제로 동작하는 것을 보았다!

5.4.5 (다시 해보는) 시각화

시작할 때 했던 대로 데이터를 차트에 그려보자. 데이터 시각화는 데이터 과학자가 해야 하는 첫 번째 작업이다. 데이터를 알려면 이런 식으로 항상 먼저 그려보기 바란다.

```
In[22]:
```

```
%matplotlib inline
```

```
from matplotlib import pyplot as plt
```

```
t_p = model(t_un, *params)  ←—— 단위를 모르는 값을 정규화하여 훈련하고 있음. 인자도 언패킹하고 있음
```

```
fig = plt.figure(dpi=600)
```

```
plt.xlabel("Temperature (°Fahrenheit)")
```

```
plt.ylabel("Temperature (°Celsius)")
```

```
plt.plot(t_u.numpy(), t_p.detach().numpy())  ←—— 알 수 없는 원본 값을 그려보고 있음
```

```
plt.plot(t_u.numpy(), t_c.numpy(), 'o')
```

이 코드에서는 인자 언패킹(argument unpacking)이라는 파이썬 기법을 활용하고 있다. *param은 params 요소를 개별 인자로 전달한다는 의미다. 파이썬에서는 리스트나 튜플로 전달하는 경우가 대부분이지만 파이토치 텐서에서는 선두 차원을 따라 분할되는 인자 언패킹을 사용할 수 있다. 예제에서 model(t_un, *params)는 model(t_un, params[0], params[1])과 동일하다.

이 코드가 그린 그래프는 그림 5.9와 같다. 우리의 선형 모델이 데이터를 표현하는 데 좋은 선택인 듯하다. 그리고 우리의 측정 값의 부정확한 모습도 그림에서 볼 수 있다. 안과에 가서 안경을 다시 맞추거나 멋진 온도계를 환불하는 게 나올지도 모르겠다.

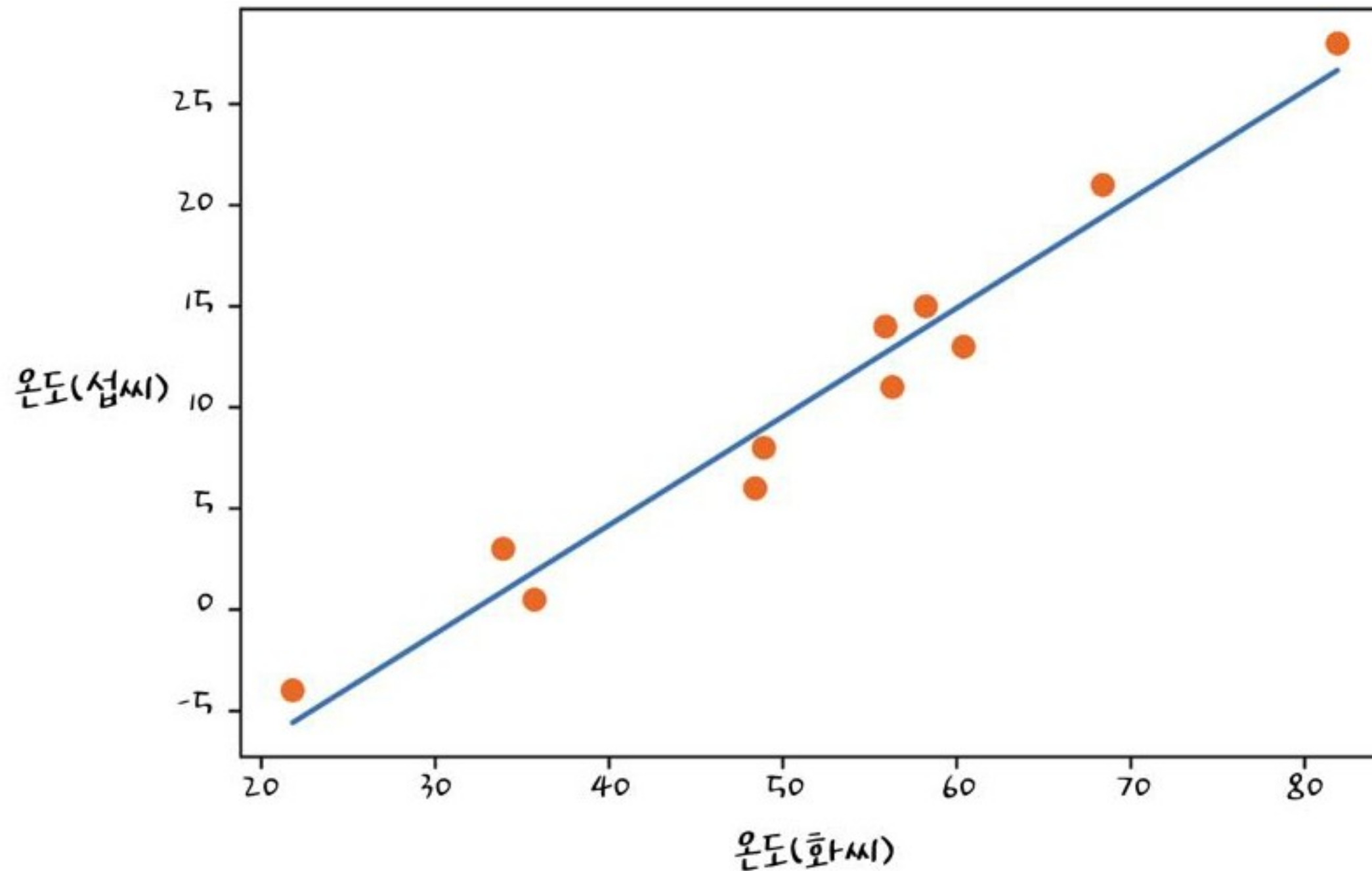


그림 5.9 입력 데이터(점)와 데이터에 맞춰진 선형 모델(선)

5.5 파이토치의 자동미분: 모든 것을 역전파하라

앞에서 작은 모형을 통해 간단한 역전파 예제를 살펴보았다. 연쇄 규칙^{chain rule}을 사용하여 미분을 역방향으로 전파하는 방법을 통해 w 와 b 를 내부 파라미터로 가지는 모델과 손실에 대한 합성 함수의 기울기를 계산했다. 이 과정의 기본 전제는 우리가 다루는 모든 함수가 해석적으로 미분 가능해야 한다는 것이다. 미분 가능한 경우라면 파라미터에 대해 우리가 ‘손실값이 변하는 비율’이라 부른 기울기를 단번에 계산해낼 수 있다.

모델이 수백만의 파라미터를 가지는 복잡한 경우라고 하더라도 미분만 가능하면 파라미터에 대해 손실값의 기울기를 계산하고 미분에 대한 해석 가능한 표현식을 작성해 단번에 계산 가능하다. 인정하건대 매우 복잡한 선형과 비선형 합성 함수의 미분에 대한 해석 가능한 표현식을 작성하는 작업은 그다지 즐겁지 않다.¹¹ 빨리 끝나지도 않는다.

11 주말을 어떻게 사용할지는 여러분의 자유다. 흥미를 느낄지도 모른다.

5.5.1 기울기 자동 계산

이러한 이유로 파이토치 자동미분`autograd` 기능과 함께 파이토치 텐서가 구세주처럼 등장한다. 3장에서 텐서가 무엇인지에 대해 개괄했고 텐서에 대해 어떤 함수를 호출할 수 있는지 알아봤다. 거기에서 언급하지 않은 한 가지 흥미로운 사실이 있다. 파이토치 텐서는 자신이 어디로부터 왔는지, 즉 어느 텐서에서 어떤 연산을 수행해서 만들어진 텐서인지 기억하고 있으며, 때문에 자연스럽게 미분을 최초 입력까지 연쇄적으로 적용해 올라갈 수 있다. 이 사실 때문에 우리는 모델에서 미분을 수동으로 도출할 필요가 없다.¹² 순방향 식이 주어지기만 하면 아무리 복잡하게 꼬여 있더라도 파이토치는 입력 파라미터와 관련해 표현식에 대한 기울기를 자동으로 제공해준다.

■ 자동미분 적용하기

이를 염두에 두고 자동미분을 사용해 우리의 온도계 보정 코드를 다시 작성하고 동작을 확인하자. 먼저 손실 함수를 모델과 함께 다시 보자.

리스트 5.1 code/p1ch5/2_autograd.ipynb

```
# In[3]:
def model(t_u, w, b):
    return w * t_u + b

# In[4]:
def loss_fn(t_p, t_c):
    squared_diffs = (t_p - t_c)**2
    return squared_diffs.mean()
```

파라미터 텐서를 다시 초기화하자.

```
# In[5]:
params = torch.tensor([1.0, 0.0], requires_grad=True)
```


■ 미분 속성 사용하기

텐서 생성자에 주어진 `requires_grad=True` 인자가 보이는가? 이 인자는 `params`에 가해지는 연산의 결과로 만들어지는 모든 텐서를 이은 전체 트리를 기록하라고 파이토치에게 요청하고 있다. 바꿔 말하면 `params`를 조상으로 두는 모든 텐서는 `params`로부터 해당 텐서가 만들어지기까지 그 사이에 있는 모든 함수에 접근할 권한을 가진다는 의미다. 이 함수들이 미분 가능한 경우 (대부분의 파이토치 텐서 연산이 가능하다), 미분값은 `params` 텐서의 `grad` 속성으로 자동 기록된다.

일반적으로 모든 파이토치 텐서는 `grad` 속성을 가지는데, 주로 값은 `None`이다.

```
# In[6]:  
params.grad is None
```

```
# Out[6]:  
True
```

이 값을 얻기 위해 해야 할 일은 텐서를 만들 때 `requires_grad`를 `True`로 지정하고, 모델을 호출해 손실값을 구한 다음, `loss` 텐서에 대해 `backward`를 호출하는 것뿐이다.

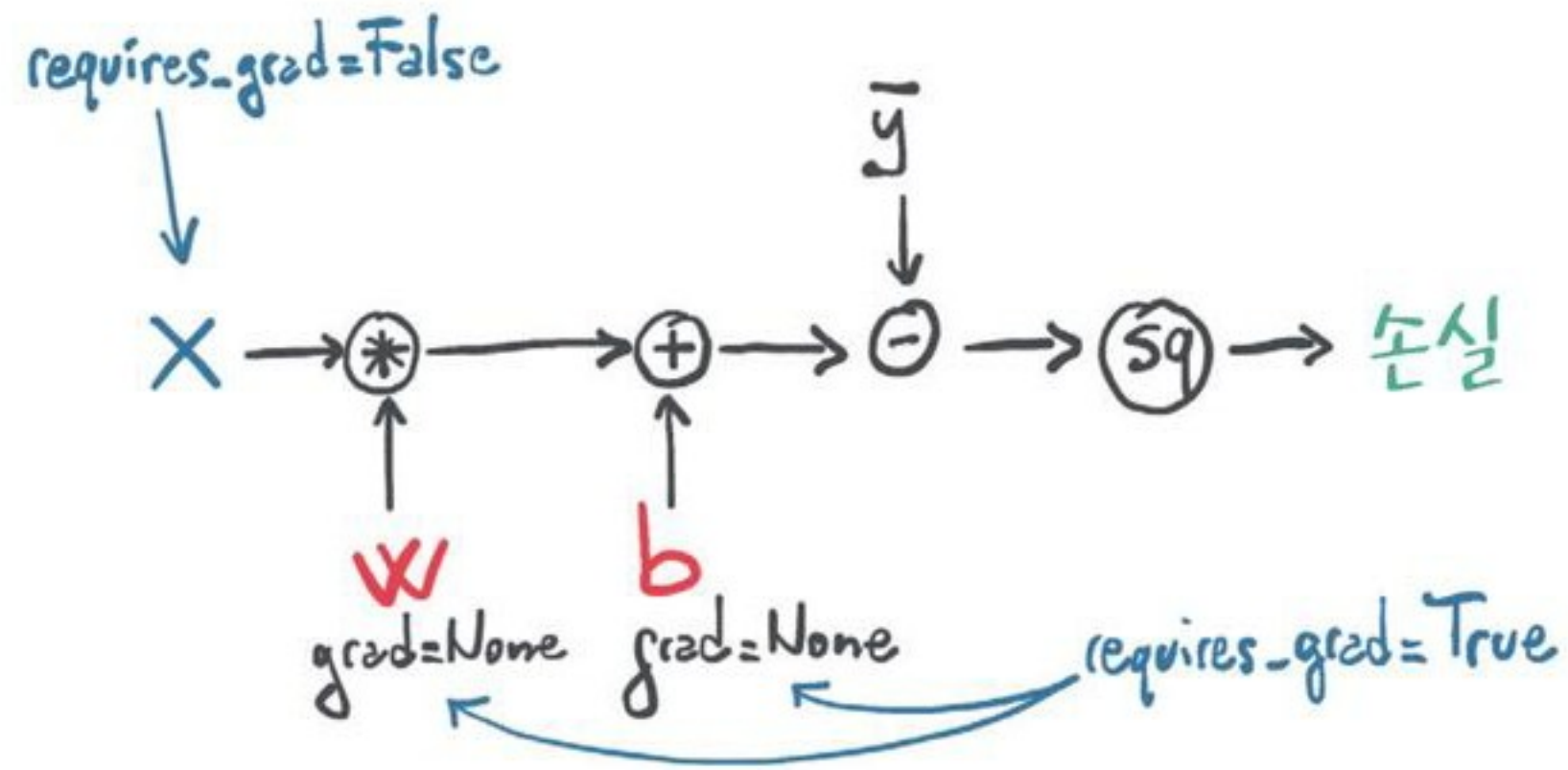
```
# In[7]:  
loss = loss_fn(model(t_u, *params), t_c)  
loss.backward()
```

```
params.grad
```

```
# Out[7]:  
tensor([4517.2969, 82.6000])
```

이제 `params`의 `grad`는 `params`의 각 요소에 대한 손실값의 미분을 포함하고 있다.

`loss`를 계산할 때, 실제 계산을 수행하는 이외에 파라미터 `w`와 `b`가 기울기를 요구하는 경우, 그림 5.10의 상단처럼 파이토치는 연산(그림에서 동그라미로 둘러싼 연산)을 노드로 하는 자동미분 그래프를 만든다. 이후 `loss.backward()`를 호출할 때, 파이토치는 그림 아래쪽에서 왼쪽 화살표 방향으로 그래프를 거꾸로 따라가면서 기울기를 계산한다.



loss.backward()

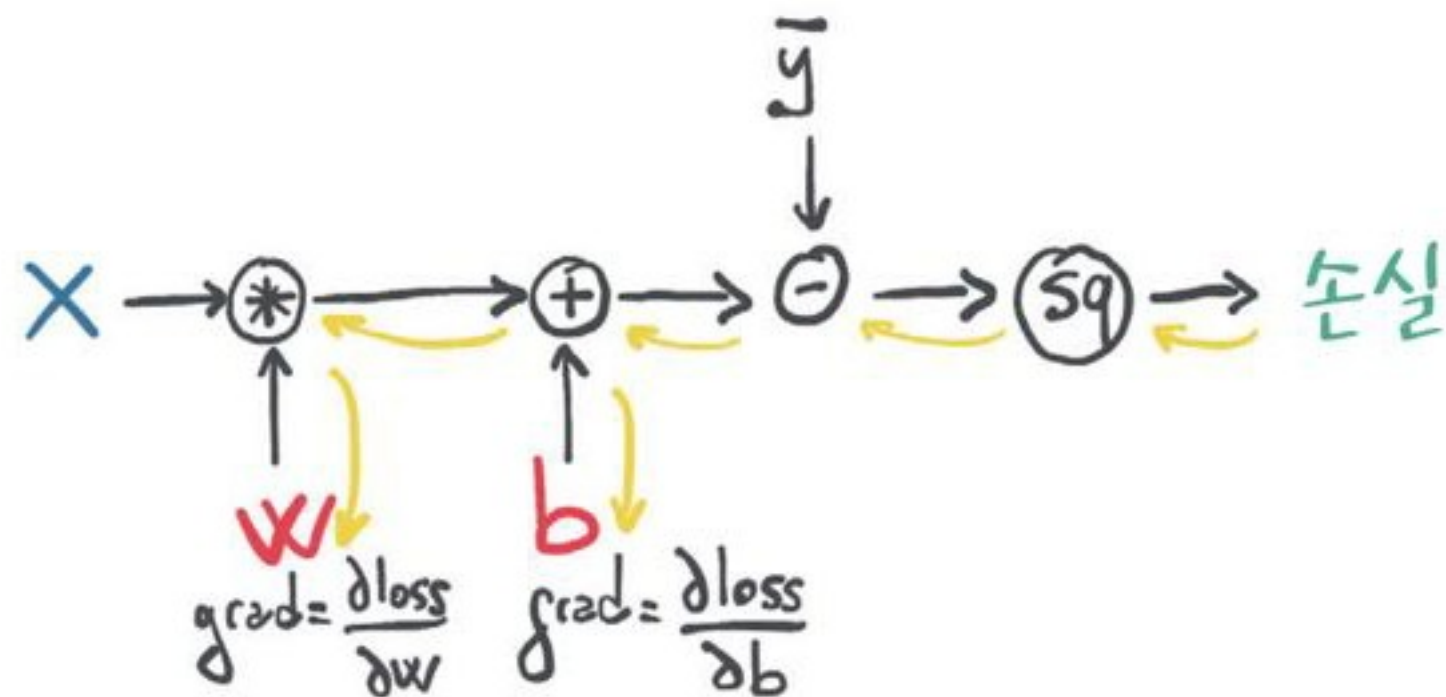


그림 5.10 자동미분 계산을 위한 모델의 순방향 그래프와 역방향 그래프

미분 함수 누적하기

텐서의 수가 몇 개이든지, 혹은 함수 합성이 얼마나 다양한지에 상관없이 `requires_grad`를 `True`로 설정할 수 있다. 이러한 경우에 파이토치는 연쇄적으로 연결된 함수들(계산 그래프`computation graph`)을 거쳐 손실에 대한 미분을 계산하고 값을 텐서(그래프의 말단 노드)의 `grad` 속성에 누적한다.

잠깐! 주의하기 바란다. 파이토치 초심자나 전문가 모두 실수하고 넘어가기 쉬운 부분이다. 방금 ‘저장`store`’이라 하지 않고 ‘누적`accumulate`’이라고 했다.

주의

`backward` 호출은 미분을 말단 노드에 누적시킨다. 따라서 파라미터 조정을 위해 사용한 후에는 기울기를 명시적으로 다시 0으로 초기화할 필요가 있다.

한 번 더 반복하자. `backward` 호출은 미분을 말단 노드에 누적한다. 따라서 앞서 `backward`가 호출되었다면 손실이 다시 계산되고 (훈련 루프에 무엇이 있든 간에) `backward`가 다시 호출되고 각 말단 노드의 기울기 값이 이전 반복문 수행 시 계산되었던 기존값에 누적되어(즉 합쳐져서) 부정확한 기울기값을 초래한다.

이를 방지하려면 각 반복문에서 명시적으로 기울기를 0으로 초기화해야 한다. 텐서 자체를 바꿔치기하는 `zero_` 메소드를 사용해서 쉽게 초기화할 수 있다.

```
# In[8]:  
if params.grad is not None:  
    params.grad.zero_()
```

노트

아마 `backward`를 호출할 때마다 기울기가 자동으로 초기화되지 않고 왜 매번 별도로 0으로 설정해줘야 하는지 궁금했을 것이다. 명시적인 초기화로 인해 복잡한 모델에서 기울기를 구할 때 더 유연하게 제어가 가능하다.

이 부분을 염두에 두고 자동미분을 적용한 훈련 코드 전체를 살펴보자.

In[9]:

```
def training_loop(n_epochs, learning_rate, params, t_u, t_c):  
    for epoch in range(1, n_epochs + 1):  
        if params.grad is not None: ← loss.backward( ) 호출 전 아무 위치에나 두면 된다.  
            params.grad.zero_()  
  
        t_p = model(t_u, *params)  
        loss = loss_fn(t_p, t_c)  
        loss.backward( )  
  
        with torch.no_grad(): ← 다음 절에서 볼 텐데 성가신 부분이긴 하지만  
                               실제로 큰 문제는 되지 않는다.  
            params -= learning_rate * params.grad  
  
        if epoch % 500 == 0:  
            print('Epoch %d, Loss %f' % (epoch, float(loss)))  
  
    return params
```


params를 조정한 코드가 기대했던 것만큼 직관적이지는 않아 보인다. 코드에서 두 가지 특징을 볼 수 있다. 첫 번째는 파이썬 with로 파라미터 조정이 no_grad 컨텍스트 안에서 이뤄지도록 둘러싼 것이다. 덕분에 파이토치 자동 미분 기능은 이 부분을 신경쓰지 않는다.¹³ 즉 순방향 그래프에 추가적인 엣지edge를 더할 필요가 없다는 의미이기도 하다. 실제로 이 코드를 실행할 때는 backward를 호출할 때 파이토치가 기록한 순방향 그래프를 사용하게 되며, 말단 노드인 params를 얻게 된다. 하지만 이제 이 상황에서 새로운 순방향 그래프의 구축을 시작하기 앞서 이 말단 노드를 변경하기를 원한다. 통상 이 부분은 5.5.2절에서 다룰 옵티마이저 안에 구현하지만 5.5.4절에서 no_grad의 실제 활용에 대해 알아볼 때 구체적으로 살펴볼 예정이다.

두 번째는 params를 덮어쓰며 조정한다. 즉 params 텐서를 계속 유지하면서 조정할 값을 뽑아내고 있다. 자동미분 사용시에는 값을 덮어쓰며 조정하지 않는다. 파이토치의 자동미분이 역방향 전달시 변수 값을 바꿔야 할 수 있기 때문이다. 지금은 자동미분 없이 동작하므로 그냥 params 텐서를 유지하는 것이 더 낫다. 새로운 텐서를 변수 이름에 할당해서 파라미터를 변경하지 않는 방식은 5.5.2절에서 옵티마이저에 파라미터를 등록할 때 매우 중요한 부분이 될 것이다.

잘 동작하는지 보자.

```
# In[10]:
training_loop(
    n_epochs = 5000,
    learning_rate = 1e-2,
    params = torch.tensor([1.0, 0.0], requires_grad=True), ← requires_grad=True가 핵심
    t_u = t_un, ← 여기서도, t_u 대신 정규화된 t_un을 사용
    t_c = t_c)
```

```
# Out[10]:
Epoch 500, Loss 7.860116
Epoch 1000, Loss 3.828538
Epoch 1500, Loss 3.092191
Epoch 2000, Loss 2.957697
Epoch 2500, Loss 2.933134
Epoch 3000, Loss 2.928648
Epoch 3500, Loss 2.927830
Epoch 4000, Loss 2.927679
Epoch 4500, Loss 2.927652
Epoch 5000, Loss 2.927647
```

```
tensor([ 5.3671, -17.3012], requires_grad=True)
```

이전에 봤던 것과 결과가 동일하다. 좋다! 우리는 수작업으로 미분 계산이 가능하지만, 이제 굳이 그렇게 할 필요는 없다.

5.5.2 골라쓰는 옵티마이저

앞의 코드에서 우리는 최적화를 위해 기본 버전의 경사 하강을 사용했고, 단순한 경우 잘 동작했다. 당연하지만 모델이 복잡해질 경우에도 수렴을 위해 사용할 수 있는 여러 가지 최적화 전략과 기법이 존재한다.

이후 몇 장에 걸쳐 더 깊게 살펴볼 예정이지만, 지금은 사용자 코드로부터 최적화 관련된 전략을 추상화시켜주는 파이토치의 방식에 대한 소개 정도만 해두자. 이 부분은 사실 이미 살펴본 훈련 루프다. 최적화 지원으로 모델에 대해 우리가 매번 모든 파라미터를 직접 조정해줘야 하는 지루한 작업을 대신 해준다. torch에는 optim 서브 모듈이 있는데 다양한 최적화 알고리즘 구현 클래스를 찾아볼 수 있다. 간단하게 추려본 최적화 알고리즘 리스트는 다음과 같다([code/p1ch5/3_optimizers.ipynb](#)).


```

# In[5]:
import torch.optim as optim

dir(optim)

# Out[5]:
['ASGD',
 'Adadelta',
 'Adagrad',
 'Adam',
 'Adamax',
 'LBFGS',
 'Optimizer',
 'RMSprop',
 'Rprop',
 'SGD',
 'SparseAdam',
 ...
]

```

모든 옵티마이저 생성자는 첫 번째 입력으로 파라미터 리스트(통상 `requires_grad`가 `True`로 설정된 파이토치 텐서)를 받는다. 옵티마이저에 전달된 파라미터는 옵티마이저 객체 내부에 유지되고, 그림 5.11에 있는 것처럼 값을 조정하고 `grad` 속성에 접근할 때 사용된다.

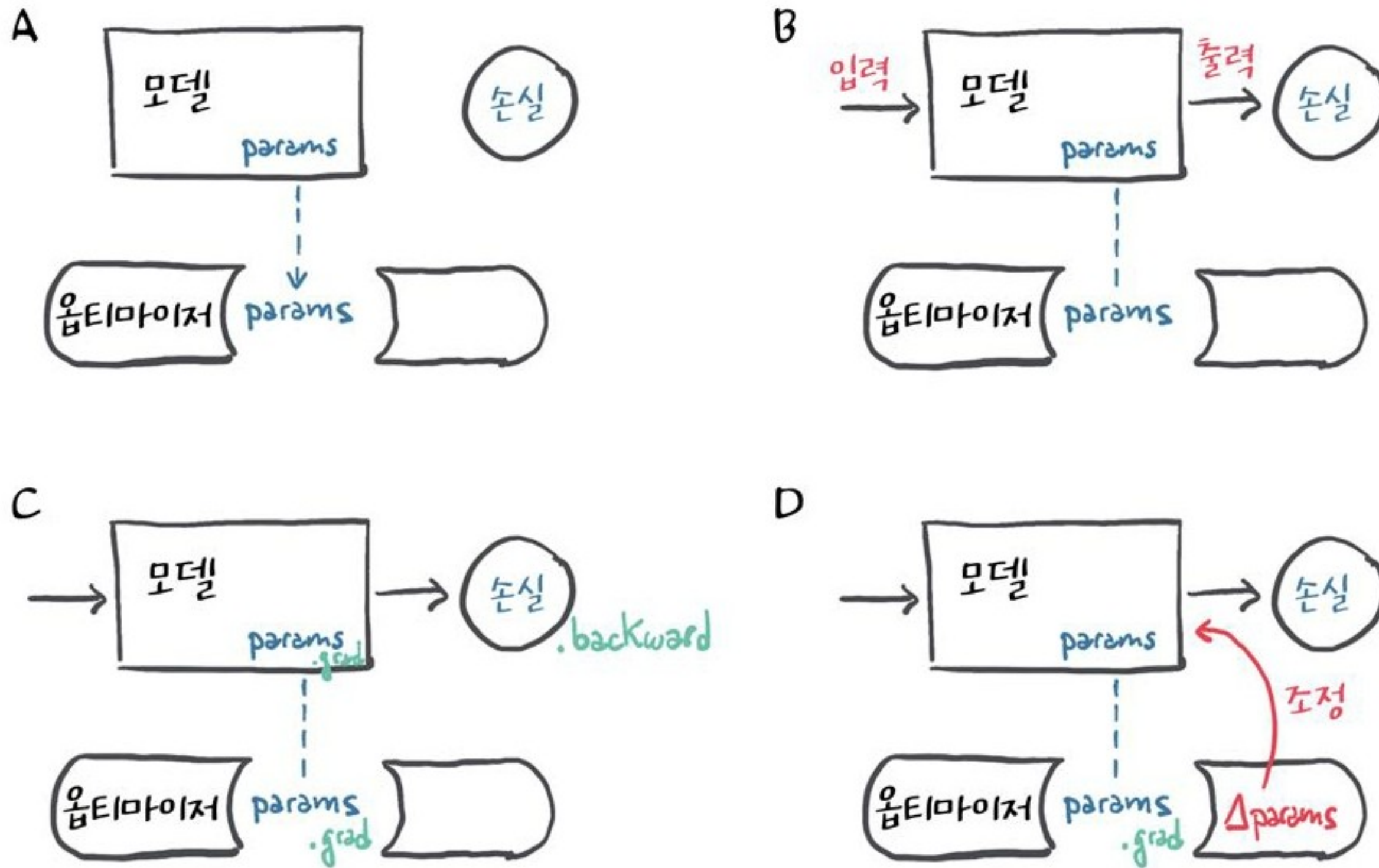


그림 5.11 (A) 옵티마이저가 파라미터 참조를 어떻게 유지하는지를 그린 것. (B) 입력으로부터 손실값이 계산된 후, (C) .backward가 호출되어 파라미터에 대해 .grad가 계산됨. (D) 이후 옵티마이저는 .grad에 접근해서 파라미터를 조정함

모든 옵티마이저는 zero_grad와 step라는 두 가지 메소드를 제공한다. zero_grad는 옵티마이저 생성자에 전달됐던 파라미터의 모든 grad 속성값을 0으로 만든다. step은 옵티마이저별로 구현된 최적화 전략에 따라 파라미터 값을 조정한다.

■ 경사 하강 옵티마이저 사용하기

params를 만들고 경사 하강 옵티마이저 인스턴스를 생성하자.

```
# In[6]:  
params = torch.tensor([1.0, 0.0], requires_grad=True)  
learning_rate = 1e-5  
optimizer = optim.SGD([params], lr=learning_rate)
```

SGD는 **확률적 경사 하강** *stochastic gradient descent*의 약자다. 실제로 순정 버전의 경사 하강과 완전히 동일하다(*momentum* 인자가 기본값인 0.0으로 설정됐을 때). SGD의 기울기는 **미니 배치** *minibatch*라고 불리는 여러 샘플 중에서 임의로 뽑은 일부에 대해 평균을 계산해 얻기 때문에 **확률적** *stochastic*이라는 말을 사용한다. 다만 옵티마이저는 손실값이 모든 샘플로부터 얻은 것(순정)인지 일부를 임의로 선택해 얻은 것(확률적)인지 모르기 때문에 알고리즘적으로는 두 경우가 동일하다.

아무튼, 이제 우리의 멋진 새 옵티마이저를 한번 돌려보자.


```
# In[7]:
t_p = model(t_u, *params)
loss = loss_fn(t_p, t_c)
loss.backward()

optimizer.step()

params

# Out[7]:
tensor([ 9.5483e-01, -8.2600e-04], requires_grad=True)
```

이제 직접 건드리지 않아도 step 호출을 통해 params가 알아서 조정된다. 우리가 손으로 직접 수행한 코드처럼 옵티마이저는 params.grad를 보고 params에서 grad에 learning_rate를 곱한 것만큼 값을 빼서 조정한다.

훈련 루프에 이제 이 코드를 넣으면 될까? 아직은 아니다! 기울기 값을 0으로 초기화하는 것을 잊지 않았는가. 앞서 작성한 코드의 루프에서 그랬던 것처럼 backward 호출 때 기울기값은 말단 노드에 계속 누적되며 경사 하강이 모든 곳에서 일어날 것이다! 적당한 곳에 `zero_grad(backward 호출 직전)`를 추가하여 루프에 넣을 준비가 된 코드는 다음과 같다.

```
# In[8]:
params = torch.tensor([1.0, 0.0], requires_grad=True)
learning_rate = 1e-2
optimizer = optim.SGD([params], lr=learning_rate)

t_p = model(t_un, *params)
loss = loss_fn(t_p, t_c)

optimizer.zero_grad()  ← 예전과 마찬가지로, 호출 위치는 여기 혹은 루프 내에서 더 위쪽도 괜찮다.
loss.backward()
optimizer.step()

params

# Out[8]:
tensor([1.7761, 0.1064], requires_grad=True)
```

완벽하다! optim 모듈이 특정 옵티마이저 구현을 얼마만큼 단순하게 만들었는지 보이는가? 이제 해야 할 일은 단순히 파라미터 리스트를 전달하고(매우 긴 심층 신경망 모델이라면 리스트 또한 매우 길어진다) 나머지 구체적인 내용은 잊어버려도 된다.

훈련 루프도 여기에 맞춰 고쳐보자.

```
# In[9]:
def training_loop(n_epochs, optimizer, params, t_u, t_c):
    for epoch in range(1, n_epochs + 1):
        t_p = model(t_u, *params)
        loss = loss_fn(t_p, t_c)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 500 == 0:
            print('Epoch %d, Loss %f' % (epoch, float(loss)))

    return params
```



```

# In[10]:
params = torch.tensor([1.0, 0.0], requires_grad=True)
learning_rate = 1e-2
optimizer = optim.SGD([params], lr=learning_rate)

training_loop(
    n_epochs = 5000,
    optimizer = optimizer,
    params = params,
    t_u = t_un,
    t_c = t_c)

# Out[10]:
Epoch 500, Loss 7.860118
Epoch 1000, Loss 3.828538
Epoch 1500, Loss 3.092191
Epoch 2000, Loss 2.957697
Epoch 2500, Loss 2.933134
Epoch 3000, Loss 2.928648
Epoch 3500, Loss 2.927830
Epoch 4000, Loss 2.927680
Epoch 4500, Loss 2.927651
Epoch 5000, Loss 2.927648

tensor([ 5.3671, -17.3012], requires_grad=True)

```



이곳의 params가 동일하지 않으면 옵티마이저는 모델이 사용하는 파라미터가 어떤 것인지 알 수 없다.

다시 앞의 것과 동일한 결과를 얻었다. 훌륭하다. 이 정도면 어떻게 경사 하강을 사용할 수 있는지 확실하게 이해했다고 봐도 된다.

■ 다른 옵티마이저 테스트하기

다른 옵티마이저를 더 테스트 해보려면 SGD 대신 아담^{Adam} 같은 다른 옵티마이저를 인스턴스로 만들면 된다. 코드의 나머지 부분은 건드리지 않아도 된다. 정말 편리하다.

아담 옵티마이저에 대해서는 구체적으로 살펴보지 않을 것이다. 간단하게 설명하면 아담은 학습률이 동적으로 변하는 좀 더 섬세한 옵티마이저다. 이 옵티마이저는 파라미터의 비율 조정에도 덜 영향받기 때문에 정규화되지 않은 원래 입력 t_u 를 사용하고 학습률을 $1e-1$ 로 올려도 지장 없이 학습을 수행한다.

```

# In[11]:
params = torch.tensor([1.0, 0.0], requires_grad=True)
learning_rate = 1e-1
optimizer = optim.Adam([params], lr=learning_rate) ← 새 옵티마이저 클래스

training_loop(
    n_epochs = 2000,
    optimizer = optimizer,
    params = params,
    t_u = t_u, ← 원래 입력인 t_u를 사용
    t_c = t_c)

# Out[11]:
Epoch 500, Loss 7.612903
Epoch 1000, Loss 3.086700
Epoch 1500, Loss 2.928578
Epoch 2000, Loss 2.927646

tensor([ 0.5367, -17.3021], requires_grad=True)

```


훈련 루프에서 유연한 부분은 옵티마이저뿐만이 아니다. 모델을 생각해보자. 동일한 데이터와 손실 함수로 신경망을 훈련할 때 단순히 model 함수만 바꿀 수도 있다. 지금 예제의 경우에는 섬씨의 화씨 변환은 선형 변환인 것을 알고 있기 때문에 큰 의미가 없지만 6장에서 다른 식으로 수행해볼 것이다. 그러면 신경망 덕분에 우리가 근사approximating하려는 함수가 어떤 형태인지조차 알 필요가 없음을 이해하게 될 것이다. 그렇게 해서 실제 과정이 매우 복잡한 비선형이라고 하더라도 신경망이 어떻게 잘 훈련하게 되는지 확인할 수 있다(2장에서 본, 이미지를 문장으로 기술하는 경우가 이에 해당한다).

이제 우리는 머신러닝의 내부 동작을 알고 필수적인 개념을 배워서 복잡한 형태의 딥러닝 모델을 훈련시킬 수 있게 됐다. 기울기를 추정하기 위한 역전파, 자동미분, 모델 가중치의 최적화를 경사 하강이나 다른 옵티마이저를 통해 수행하는 것을 배웠다. 실제로는 별 것 없다. 나머지는 범위가 넓긴 하지만 대부분 빈칸을 채워 넣기다.

다음으로 우리는 훈련 샘플을 어떻게 부분 부분으로 나눌지 생각해볼 것이다. 이를 통해 자동미분을 더 잘 다루는 법을 알게 될 것이다.

5.5.3 훈련, 검증, 과적합

요하네스 케플러로부터 배운 것 중 아직 언급하지 않은 것이 있는데 기억나는가? 케플러는 데이터의 일부를 따로 떼어놓고 나중에 자신의 모델을 별도의 독립된 관찰 표본으로 검증한 바 있다. 매우 중요한 부분인데, 신경망에서 사용하는 모델이 잠정적으로 모든 형태로 근사할 수 있는 함수를 가진 경우에 더더욱 중요하다. 바꿔 말하면 적응성이 높은 모델은 많은 파라미터를 사용하여 주어진 데이터 포인트에 대해 손실을 최소화하도록 학습되지만 이 상태가 데이터 포인트 사이나 데이터 포인트에서 벗어난 경우에 대해서도 잘 동작할 수 있는지 보장할 수는 없다. 우리는 옵티마이저에게 주어진 데이터 포인트에 대한 손실을 최소화하라고 요청하고 있다. 이 때문에 우리가 손실을 구하거나 음의 기울기를 따라 내려갈 때 사용했던 데이터와는 다른 별개의 데이터를 사용하면 기대했던 것보다 높은 손실값을 얻을 수 있다. 이런 현상을 과적합 overfitting이라 한다.

과적합을 방지하기 위해 우리가 대응할 수 있는 방법 중 하나는 과적합이 일어날 수 있음을 먼저 인지하는 것이다. 그림 5.12와 유사하게 케플러가 1600년에 그렇게 했던 것처럼 데이터에서 일부를 따로 떼어놓고(검증셋 validation set으로 부른다) 남은 데이터(훈련셋 training set)로 모델을 맞춰간다. 모델을 맞춰가면서 이제 손실을 한 번은 훈련셋에 대해 구하고, 한 번은 검증셋에 대해 구할 수 있다. 모델을 데이터에 맞추는 작업이 잘 끝났는지 판단하려면, 두 가지 결과를 살펴보면 된다!

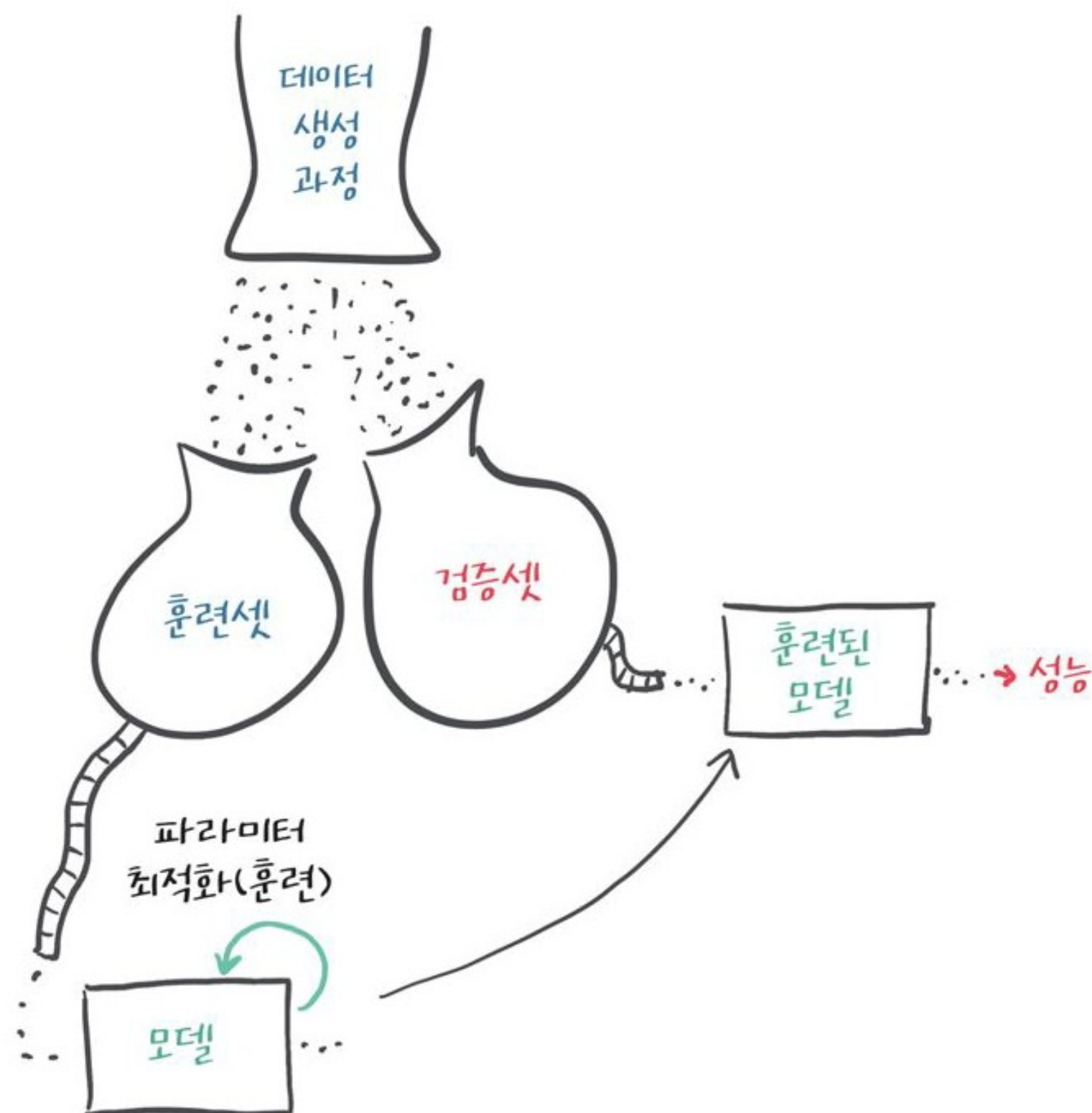


그림 5.12 데이터 생성 과정과 훈련 데이터 및 검증 데이터를 준비하는 과정에 대한 개념도

■ 훈련 손실 평가하기

훈련 손실값은 모델이 훈련셋에 얼마나 잘 맞춰졌는지, 즉 모델이 데이터에 있는 필요한 정보를 찾아 처리하기에 충분한지를 알려준다. 만일 앞서의 온도계가 온도를 로그 범위로 측정하는 경우라면, 다소 능력이 떨어지는 우리의 선형 모델은 측정에 기반한 데이터에 맞춰질 기회를 얻지 못했을 것이고 섭씨로의 변환도 제공할 수 없었을 것이다. 그랬다면 훈련 손실(훈련 루프에서 우리가 출력하던 손실값)은 0에 가까워지기 전에 줄어들기를 멈췄을 것이다.

심층 신경망은 충분한 수의 뉴런, 즉 파라미터가 제공된다면 매우 복잡한 함수에도 근사할 수 있다. 파라미터 수가 적다면 근사할 수 있는 함수는 더욱 단순한 모양에 그치고 만다. 그렇기 때문에, [규칙 1] 훈련 손실이 줄어들지 않는다면 데이터에 비해 모델이 너무 단순할 가능성이 있다. 아니라면 출력을 설명할 만한 데이터가 아닐 수도 있다. 앞에서 점원이 우리에게 온도계라 하고 기압계를 팔았다면 기압으로 온도를 예측하려 했을지도 모른다. 이것은 퀘벡에서 만든 신경망 아키텍처(www.umontreal.ca/en/artificialintelligence)라도 쉽지 않은 일이다.

■ 검증셋으로 일반화하기

검증셋으로 돌아가보자. 만일 검증셋에 대한 손실이 훈련셋에서처럼 줄어들지 않는 것으로 평가된다면 모델이 훈련을 통해 바라본 샘플에 대해서는 맞춰지며 발전하지만 훈련셋을 벗어난 샘플에 대해서는 일반화되지 않는다는 의미다. 따라서 훈련 때에는 보지 않았던 데이터 포인트에 대해 평가하면 손실 함수의 값은 높게 나와버린다. [규칙 2] 훈련 손실과 검증 손실 사이에 차이가 발생한다면 과적합하는 상황인 것이다.

온도계 샘플을 통해 이 상황을 좀 더 자세히 살펴보자. 선형 함수보다 좀 더 복잡한 함수를 사용하여 데이터에 맞추려 했다고 가정하자. 예를 들면 세세하게 구분할 수 있는 다항식이나 큰 신경망 같은 것 말이다. 이러면 그림 5.13처럼 각 데이터 포인트를 따라 구불구불하게 길을 만들어 손실값이 0에 매우 가까워지는 모델이 될 수 있다. 모든 데이터 포인트에 대해 손실이 증가하지 않는 상황이기 때문에, 훈련 데이터보다 떨어져 있는 입력에 대해서는 잘 처리할 방법이 없다.

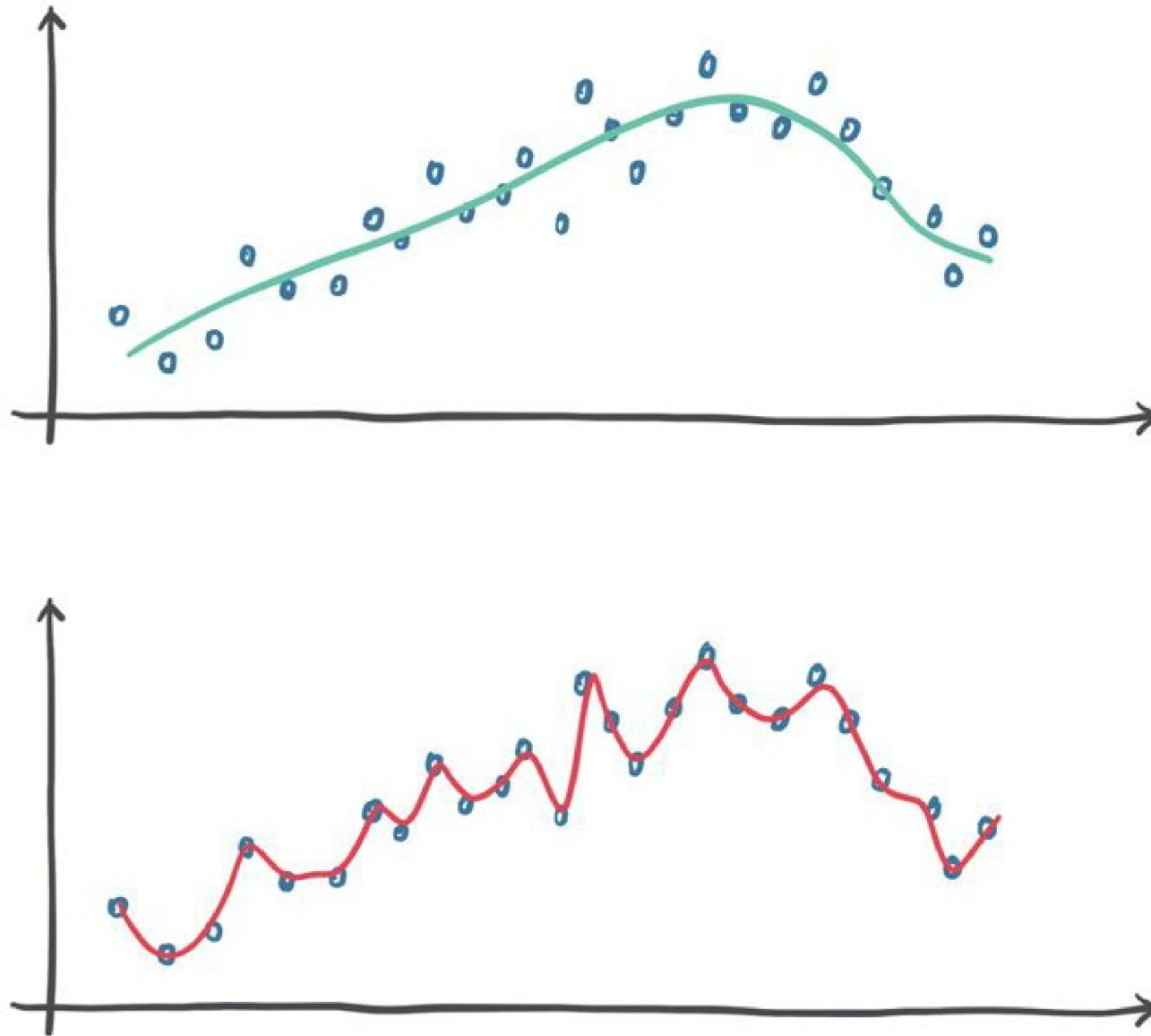


그림 5.13 다소 극단적인 과적합의 예

개선 방법은 없을까? 좋은 질문이다. 앞서 언급한 것처럼 과적합은 데이터에 종속된 모델의 동작이 우리가 근사하는 과정이 실제 문제 해결에 적합한지를 판단하는 데 하나의 장애물로 작용한다. 이를 개선하려면 먼저 처리 과정에서 충분한 데이터가 주어졌는지를 확인해야 한다. 만일 사인과 형태를 가진 저빈도의 규칙적인 샘플링으로 데이터를 모았다면 이 데이터에 모델을 맞추기는 매우 어려울 것이다.

데이터 포인트를 충분히 모았다면 다음으로 확인해야 할 사항은 모델이 각 훈련 데이터에 대해 적합한 수준으로 맞춰질 수 있는지 여부다. 여러 방법이 있는데 그중 하나는 손실 함수에 **페널티 항** *penalization term*을 두어 모델의 적합 *fitting*이(데이터에 가까워지는 과정이) 더 천천히 그리고 부드럽게 만들어지도록 하는 법이다. 다른 방법으로는 입력 샘플에 노이즈를 더하는 것인데 각 훈련 데이터 사이에 새로운 데이터 포인트를 인위적으로 만들고, 모델로 하여금 이 데이터에도 맞춰지도록 만드는 것이다. 그 외에도 다양한 방법이 있지만, 이 둘과 유사하다. 일단 우리가 처음 시도해 볼 만한 것으로는 모델의 단순화다. 단순한 모델은 복잡한 모델과 달리 훈련 데이터에 완벽하게 맞춰지지 않는 못하지만, 데이터 사이에 존재하는 값을 적절하게 맞추출 가능성이 높다.

이 지점에서 상호 절충안이 필요하다. 한편으로는 모델이 훈련을 위한 학습셋에 적합해질 만큼 충분한 능력을 갖춰야 하며, 다른 한편으로는 모델이 과적합을 방지할 수 있어야 한다. 그래서 파라미터 측면에서 적합한 신경망 사이즈를 고르는 것은 두 단계로 진행한다. 먼저 적합될 때까지 사이즈를 늘리다가, 과적합하지 않을 때까지 규모를 줄이는 것이다.

이에 관해서는 12장에서 좀 더 살펴보게 될 텐데, 결국 우리 삶도 적합과 과적합 사이를 오갈 뿐임을 느낄 수 있을 것이다. 이제 예제로 돌아가서 데이터를 어떻게 훈련셋과 검증셋으로 나눌지 생각해보자. 먼저 t_u 와 t_c 를 같은 식으로 섞어^{shuffling} 만들어진 텐서를 두 개로 나누자.

■ 데이터셋 나누기

randperm 함수는 색인 순열을 찾기 위해 텐서 요소를 섞었을 때의 인덱스 리스트를 반환한다.

```
# In[12]:  
n_samples = t_u.shape[0]  
n_val = int(0.2 * n_samples)  
  
shuffled_indices = torch.randperm(n_samples)  
  
train_indices = shuffled_indices[:-n_val]  
val_indices = shuffled_indices[-n_val:]  
  
train_indices, val_indices  ← 랜덤이기 때문에, 독자 여러분이 실행했을 때는 값이 다를 수도 있음  
  
# Out[12]:  
(tensor([9, 6, 5, 8, 4, 7, 0, 1, 3]), tensor([ 2, 10]))
```


이제 인덱스 텐서를 얻었으니 데이터 텐서로부터 훈련셋과 검증셋을 만들어보자.

```
# In[13]:  
train_t_u = t_u[train_indices]  
train_t_c = t_c[train_indices]  
  
val_t_u = t_u[val_indices]  
val_t_c = t_c[val_indices]  
  
train_t_un = 0.1 * train_t_u  
val_t_un = 0.1 * val_t_u
```

훈련 루프 자체는 바꿀 것이 없다. 각 에포크마다 부가적으로 검증셋에 대한 손실을 계산해서 과적합하고 있는지를 파악하는 부분을 추가한다.

```
# In[14]:
```

```
def training_loop(n_epochs, optimizer, params, train_t_u, val_t_u,
                  train_t_c, val_t_c):
    for epoch in range(1, n_epochs + 1):
        train_t_p = model(train_t_u, *params)
        train_loss = loss_fn(train_t_p, train_t_c)

        val_t_p = model(val_t_u, *params)
        val_loss = loss_fn(val_t_p, val_t_c)

        optimizer.zero_grad()
        train_loss.backward()
        optimizer.step()

        if epoch <= 3 or epoch % 500 == 0:
            print(f"Epoch {epoch}, Training loss {train_loss.item():.4f}, "
                  f" Validation loss {val_loss.item():.4f}")

    return params
```

이 부분은 입력이 train_*인지 혹은 val_*이지만 다르다.

← 검증 데이터로는 학습하면 안 되므로 val_loss.backward()가 없다.

```
# In[15]:
params = torch.tensor([1.0, 0.0], requires_grad=True)
learning_rate = 1e-2
optimizer = optim.SGD([params], lr=learning_rate)

training_loop(
    n_epochs = 3000,
    optimizer = optimizer,
    params = params,
    train_t_u = train_t_un,
    val_t_u = val_t_un,
    train_t_c = train_t_c,
    val_t_c = val_t_c)

```

SGD를 사용하므로 정규화된 입력을 사용함

```
# Out[15]:
Epoch 1, Training loss 66.5811, Validation loss 142.3890
Epoch 2, Training loss 38.8626, Validation loss 64.0434
Epoch 3, Training loss 33.3475, Validation loss 39.4590
Epoch 500, Training loss 7.1454, Validation loss 9.1252
Epoch 1000, Training loss 3.5940, Validation loss 5.3110
Epoch 1500, Training loss 3.0942, Validation loss 4.1611
Epoch 2000, Training loss 3.0238, Validation loss 3.7693
Epoch 2500, Training loss 3.0139, Validation loss 3.6279
Epoch 3000, Training loss 3.0125, Validation loss 3.5756
```

```
tensor([ 5.1964, -16.7512], requires_grad=True)
```


훈련셋과 검증셋은 동일한 양이 아니다. 검증셋은 매우 작기 때문에, 검증셋에서의 손실값은 의미 있을 수준 정도다. 결과에서 모든 경우에 수십 배 이상은 아니더라도 그래프상에서 검증 손실은 훈련 손실보다 크다는 사실을 기억하자. 그리고 통상적으로 모델이 훈련셋에 대해 더 잘 동작하는 것이 정상이다. 훈련셋에 대해 파라미터가 만들어지기 때문이다. 훈련의 목표는 훈련 손실과 검증 손실이 둘 다 줄어드는 형태가 되는 것이다. 이상적으로는 검증 손실이 훈련 손실에 매우 가깝게 다가가면서 두 값이 거의 같아지는 것인데 이 형태는 모델이 데이터에 대해 일반화된 학습을 지속적으로 진행하고 있는 것으로 볼 수 있다. 그림 5.14에서 C가 이상적인 형태이고 D는 그런대로 받아줄 만한 수준이지만 A는 전혀 학습이 되지 않고 있으며 B는 과적합이 발생하는 것을 볼 수 있다. 의미 있는 과적합 예제들은 12장에서 살펴보자.

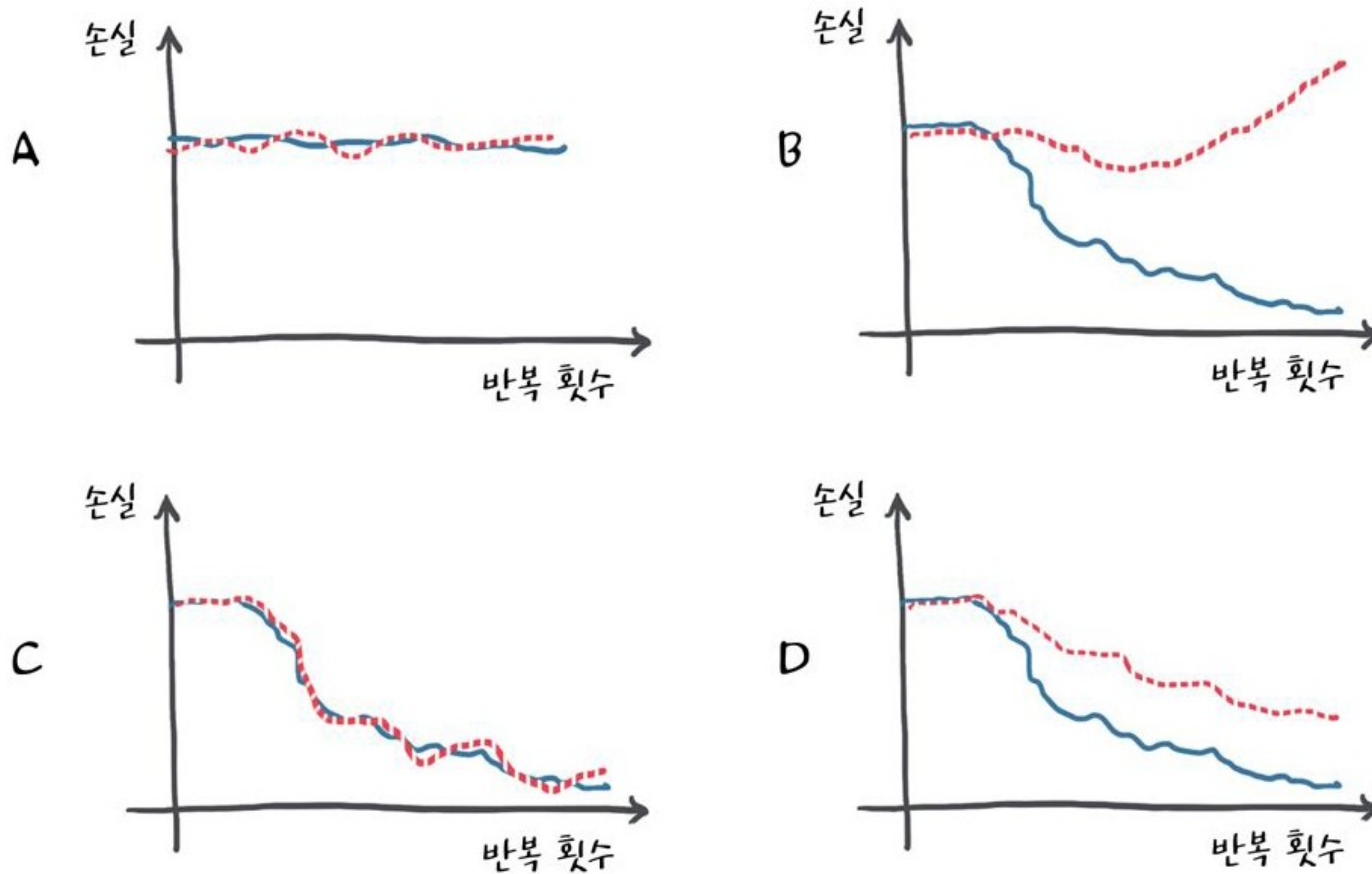


그림 5.14 훈련 손실(실선)과 검증 손실(점선)로 보는 과적합 시나리오. (A) 훈련과 검증 손실 모두 줄어들지 않는다. 데이터에서 충분한 정보가 발견되지 않거나 모델이 훈련에 필요한 용량을 갖추지 못한 경우다. (B) 훈련 손실은 줄어드는 데 반해 검증 손실은 늘어나는 경우로 과적합이다. (C) 훈련과 검증 손실이 나란히 줄어든다. 과적합의 경계에 닿지 않는 한 모델의 성능은 계속 좋아질 수 있다. (D) 훈련과 검증 손실이 비슷한 경향을 보이지만 절댓값에 차이가 있다. 과적합이 제어되고 있는 상황이다.

5.5.4 자동미분의 주의사항과 자동미분 끄기

앞의 훈련 루프에서 `train_loss`에 대해서만 `backward`를 호출한다. 따라서 역전파는 훈련셋에 대한 오차만 전파하게 된다. 검증셋은 훈련에서 사용하지 않았던 데이터에 대해 모델의 출력값 정확도를 독립적으로 평가하는 데 사용하고 있다.

호기심 많은 독자라면 이쯤에서 질문이 있을 것이다. 모델이 두 번 평가되고 있다(한 번은 `train_t_u`, 한 번은 `val_t_u`에 대해). 그러고 나서 `backward`가 호출된다. 이러면 자동미분이 이상해지지 않을까? 검증셋을 거치며 만들어진 값들에 대해 `backward`가 영향받지 않을까?

다행히도 그렇지 않다. 훈련 루프의 첫 줄은 `train_t_p`를 만들기 위해 `train_t_u`에 대해 모델을 평가한다. 그러고 나서 `train_t_p`로부터 `train_loss`가 평가된다. 때문에 `train_t_u`에서 `train_t_p`로, 그리고 `train_loss`로 이어지는 연산 그래프가 만들어진다. `model`이 `val_t_u`에 대해 평가될 때에는 `val_t_p`와 `val_loss`를 만든다. 이 경우에 `val_t_u`에서 `val_t_p`를 거쳐 `val_loss`로 가는 별도의 연산 그래프가 만들어질 것이다. 그림 5.15처럼 `model`과 `loss_fn` 함수에 대해 별개의 텐서가 각기 다른 연산 그래프를 만들기 때문에 괜찮다.

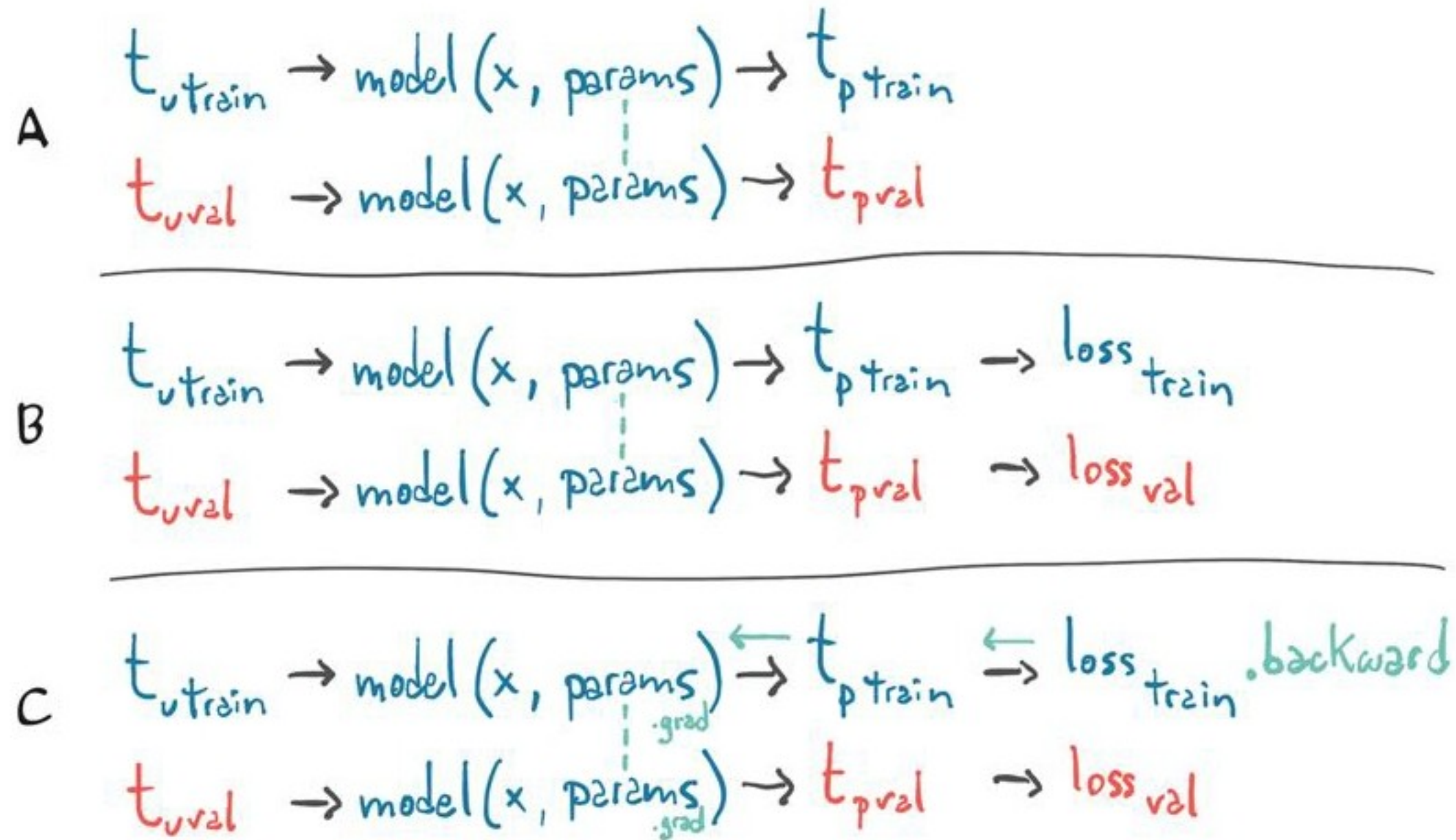


그림 5.15 둘 중 하나에 .backward가 호출될 때, 그래프를 통과해 두 손실 값과 함께 기울기가 전파되는 모습을 보여주는 다이어그램

두 그래프가 공유하는 유일한 텐서는 파라미터 텐서다. train_loss에 대해 backward를 호출할 때에는 첫 번째 그래프에 대해 backward를 수행한다. 즉 train_t_u에서 만들어진 연산 그래프의 파라미터에 대해 train_loss의 미분을 누적한다.

만일 실수로 `val_loss`에 대해 `backward`를 호출하면 동일한 말단 노드인 파라미터 텐서에 대해 `val_loss`의 미분을 누적시켜버리게 된다. 기울기를 명시적으로 0으로 초기화하지 않으면 `backward` 호출 때마다 기울기가 누적되므로 `zero_grad`를 사용했던 것을 기억하는가? 이것도 비슷한 경우다. `val_loss`에서 `backward`를 호출하면 `train_loss.backward()` 호출로 값이 기록된 `params` 텐서에 추가로 기울기를 누적해버린다. 이 경우에, 기울기가 양쪽에 의존하므로 훈련셋과 검증셋 모두에 대해 모델을 훈련시키는 셈이다. 꽤 흥미로운 상황이다.

추가적으로 생각해볼 부분이 하나 더 있다. `val_loss`에 대해 `backward`를 호출하지 않으면서 처음에 연산 그래프를 만드는 이유는 무엇일까? 계산을 추적하지 않고서 일반 함수처럼 `model`과 `loss_fn`을 호출하면 되지 않을까? 그럼에도 최적화된 자동미분 그래프 생성에는 검증 과정에는 생략해도 되는 비용이 들어간다. 파라미터가 수백만 개에 이르는 모델이라면 비용도 더 커진다.

이를 위해서 파이토치는 `torch.no_grad` 컨텍스트 관리자¹⁴를 통해 필요하지 않을 때 자동미분을 끌 수 있게 해준다. 지금 다루는 문제는 작기 때문에 이런 조정을 통해 속도나 메모리 사용 절감 등의 유의미한 이득을 보기는 어렵다. 하지만 큰 모델의 경우는 다르다. `val_loss` 텐서의 `requires_grad` 속성을 통해 값이 원하는 대로 설정되었는지 확인할 수 있다.

```
# In[16]:
```

```
def training_loop(n_epochs, optimizer, params, train_t_u, val_t_u,
                  train_t_c, val_t_c):
```

```
    for epoch in range(1, n_epochs + 1):
```

```
        train_t_p = model(train_t_u, *params)
```

```
        train_loss = loss_fn(train_t_p, train_t_c)
```

```
        with torch.no_grad(): ←—— 컨텍스트 관리자
```

```
            val_t_p = model(val_t_u, *params)
```

```
            val_loss = loss_fn(val_t_p, val_t_c)
```

```
            assert val_loss.requires_grad == False
```

이 블록 내에서 `requires_grad`가 `False`로 설정을 강제한다는 상황을 점검한다.

```
        optimizer.zero_grad()
```

```
        train_loss.backward()
```

```
        optimizer.step()
```


`set_grad_enabled`를 통한 컨텍스트 설정으로는 autograd를 켜지 혹은 끌지를 제어할 수 있다. 불리언 표현으로는 현재 훈련 모드인지, 추론 모드인지를 나타낸다. 예를 들어 `calc_forward` 함수를 정의하고 데이터를 입력받아 `model`과 `loss_fn`을 실행하되, `is_train` 인자에 따라 자동미분의 사용 여부를 설정할 수도 있다.

```
# In[17]:
def calc_forward(t_u, t_c, is_train):
    with torch.set_grad_enabled(is_train):
        t_p = model(t_u, *params)
        loss = loss_fn(t_p, t_c)
    return loss
```

5.6 결론

이 장은 기계가 예제를 통해 어떻게 학습하는가라는 질문으로 시작했다. 그리고 이 장의 나머지 부분은 모델을 데이터에 맞추기 위한 최적화 메커니즘에 대해 설명했다. 또한 불필요하며 복잡한 요소는 제거하여 동작하는 부분 부분을 볼 수 있도록 단순한 모델을 선정했다.

이제 궁금한 기본적인 사항들은 충분히 살펴보았으므로, 다음 6장에서는 본격적인 주제를 시작하려 한다. 데이터에 맞추기 위해 신경망을 사용하는 과정이다. 동일한 온도계 문제를 풀지만 `torch.nn`이 제공하는 강력한 도구를 사용할 예정이다. 지금까지 5장에서 문제를 풀기 위해 사용했던 것과 동일한 접근 방식이지만, 파이토치 사용에 대해 더 많이 설명하게 될 것이다. 사실 온도계 문제 해결에 신경망까지는 필요없지만 이 과정을 통해 여러분은 신경망 훈련에 무엇이 필요한지를 더 간단히 이해할 수 있게 될 것이다.

5.8 핵심 요약

- 선형 모델은 모델 적합을 위한 가장 합리적이면서도 단순한 모델이다.
- 볼록 함수 최적화 기법은 선형 모델에 사용 가능하지만 신경망에 적용할 수 있을 만큼 일반화하기는 어렵다. 이 때문에 파라미터 값의 추정을 위해서는 통계적 경사 하강을 사용한다.
- 딥러닝은 특정 문제를 풀기 위한 엔지니어링이 아닌 일반적인 경우에 사용하는 모델로서, 스스로 적응하여 문제 풀이에 최적화시키는 특징이 있다.
- 학습 알고리즘은 관찰을 통해 모델 파라미터를 최적화하는 수준에 다르다. 손실 함수는 작업 수행시 발생하는 오차를 측정하는 수단으로 기대값과 실제 측정값 사이의 오차를 의미한다. 여기에서의 목표는 손실 함수를 최대한 작게 만드는 것이다.
- 모델 파라미터에 대한 손실 함수의 변화율은 손실을 줄이는 방향으로 동일한 파라미터를 조정하는 데 사용할 수 있다.

- 파이토치의 optim 모듈은 바로 사용 가능한 옵티마이저를 제공하여 손실 함수를 최소화하는 방향으로 파라미터를 조정한다.
- 옵티마이저는 최종 출력에 대한 기여도에 따라 각 파라미터에 적용할 기울기를 계산하기 위해 파이토치의 자동미분 기능을 사용한다. 사용자는 복잡한 순방향 전달 중에 필요한 작업을 동적 연산 그래프에 맡길 수 있다.
- `with torch.no_grad()`: 같은 컨텍스트 관리자는 자동미분 기능의 동작을 제어하는 데 사용할 수 있다.
- 데이터는 훈련 샘플과 검증 샘플로 분리해 둔다. 이를 통해 훈련에 사용하지 않았던 데이터로 모델을 평가할 수 있다.
- 과적합은 모델이 훈련셋에 대해서는 발전하는 반면, 검증셋에 대해서는 떨어지는 경우를 말한다. 주로 모델이 일반화된 학습을 수행하는 대신 훈련셋의 출력을 암기하는 식으로 전개되기 때문에 발생한다.